



Leibniz
Universität
Hannover



GOTTFRIED WILHELM LEIBNIZ UNIVERSITÄT
HANNOVER
FACULTY OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Interactive Knowledge Extraction: A Human-in-the-Loop Approach for PDF Structuring and Knowledge Graph Integration

Interaktive Wissensextraktion: Ein nutzerzentrierter Ansatz zur
PDF-Strukturierung und Integration in Wissensgraphen

*A thesis submitted in fulfillment of the requirements for the
Master of Science in Computer Science*

BY

Sushant Aggarwal

Matriculation number: 10065113

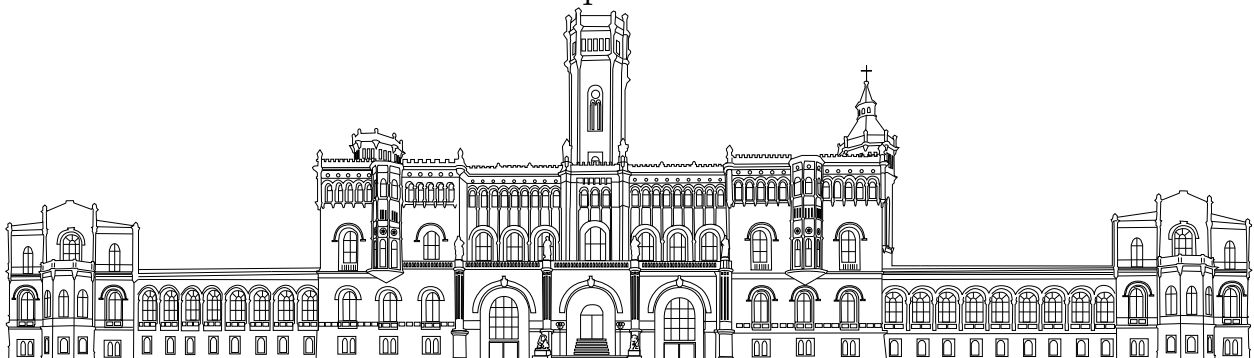
E-mail: sushant.aggarwal@stud.uni-hannover.de

First evaluator: Prof. Dr. Sören Auer

Second evaluator: Dr. Oliver Karras

Supervisor: Lena John

08. April 2026



Declaration of Authorship

I, Sushant Aggarwal, declare that this thesis titled, ‘Interactive Knowledge Extraction: A Human-in-the-Loop Approach for PDF Structuring and Knowledge Graph Integration’ and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- I used Claude and Gemini to assist with content creation and implementation, but I have checked all content for accuracy and stand behind it with my name.

Sushant Aggarwal

Signature: _____

Date: _____

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Lena John and Dr. Oliver Karras. I am deeply grateful for their unwavering support, expert guidance, and the profound insights they shared throughout this process. Their mentorship has made this research a truly inspiring experience.

I am also indebted to the TIB for providing the essential resources required to complete this research. I would like to thank the various members of the TIB who offered their valuable feedback, which helped shape the final direction of my work.

Finally, I want to thank my family for their constant encouragement and support during the completion of this thesis. I could not have done this without them.

Abstract

Interactive Knowledge Extraction: A Human-in-the-Loop Approach for PDF Structuring and Knowledge Graph Integration

CONTEXT: The digital transformation of scientific literature relies heavily on the creation of structured Knowledge Graphs (KGs). Transitioning dense research papers into machine-readable data requires mapping complex text to strict curation templates for comparability. However, manual curation is a highly demanding and time-consuming bottleneck that requires specialized domain expertise and cannot scale with the growing volume of publications. While Large Language Models (LLMs) offer rapid extraction capabilities, applying them in a fully automated, “black-box” manner often results in hallucinated evidence, loss of structural context, and untraceable answers, making them unreliable for rigorous scientific documentation.

OBJECTIVE: To overcome these limitations, this thesis proposes a steerable Human-in-the-Loop (HITL) extraction framework built upon a flexible, model-agnostic architecture to prevent vendor lock-in. The primary objective is to develop an intelligent helper tool that assists human curators in finding specific values within complex documents. Rather than attempting to fully automate data entry, this approach ensures that the human expert remains firmly in control. It utilizes the AI to accelerate the search process while empowering the curator to seamlessly accept, refine, or reject generated suggestions and actively steer the model’s reasoning through natural language feedback.

METHOD: To structure the curation task, the proposed method utilizes pre-defined, hierarchical questionnaires derived from strict scientific schemas. Rather than allowing the AI to generate unstructured text, these questionnaires act as a rigid framework, constraining the model to locate and extract specific, targeted data points. To generate highly accurate suggestions for these fields, the framework employs targeted context engineering, dynamically supplying the LLM with relevant source text, structural awareness of the schema (such as parent and sibling question relationships), and iterative user feedback. To resolve the lack of traceability in standard LLMs, the method introduces a multi-stage spatial mapping pipeline paired with a two-pass hybrid search algorithm. This mathematically anchors the AI’s abstract semantic output back to exact geometric nodes within the physical PDF document. Finally, to rigorously validate the underlying logic, an automated, headless evaluation pipeline was developed to benchmark the AI’s zero-shot extraction performance against the KG-EmpIRE dataset—a gold-standard collection of manually annotated literature—by applying standard evaluation metrics.

RESULTS: The empirical evaluation revealed that dynamically injecting struc-

tural context significantly improves the LLM’s logical reasoning, particularly for extracting dependent variables. However, fully autonomous extraction faced severe bottlenecks due to inherent model biases—such as affirmative biases in boolean questions, semantic confusion in single-select variables and “lazy generation” in multi-select arrays—as well as strict automated evaluation penalties for text length. Crucially, when evaluating the model’s ability to generate multiple viable extraction pathways, the context-aware pipeline successfully placed the correct answer within its top three suggestions in 77.9% of all interactions.

CONCLUSION: The findings demonstrate that fully automated LLM pipelines remain fundamentally unsafe for rigorous structured literature curation due to semantic ambiguities and formatting unpredictability. However, the high Top-3 accuracy strongly validates the proposed architecture. By framing the AI as a targeted “evidence hunter” rather than an autonomous decision-maker, the curator-focused HITL assistant effectively neutralizes foundational model flaws. This collaborative workflow ensures strict scientific rigor while drastically accelerating the manual data curation process.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goal	4
1.2.1	Enabling Granular and Steerable Human-AI Interaction . . .	5
1.2.2	Establishing Verifiable Provenance to Ensure Data Trust . .	5
1.2.3	Ensuring Cross-Domain Generalizability and Architectural Longevity	6
1.3	Structure	6
2	Background	8
2.1	Semantic Knowledge Infrastructures for Scientific Research	8
2.1.1	Knowledge Graphs	8
2.1.2	Research Knowledge Graphs	9
2.1.3	ORKG	10
2.1.4	KG-EmpIRE	10
2.1.5	EmpiRE-Compass	11
2.2	Large Language Models in Scientific Knowledge Extraction	11
2.2.1	Applications in Scholarly Research	12
2.2.2	Limitations and the Necessity of Human-in-the-Loop	12
2.3	Human-in-the-Loop (HITL)	12
2.3.1	Advantages and Disadvantages	13
2.3.2	Application Areas in High-Stakes Domains	13
2.3.3	Suitability of HITL for the Current Research	13
3	Related Work	15
3.1	Evolution of Knowledge Extraction Paradigms	15
3.1.1	Traditional Rule-Based and Statistical Machine Learning Systems	16
3.1.2	Domain-Specific Pre-Trained Language Models (PLMs) . . .	17
3.1.3	Generative Large Language Models (LLMs)	18
3.2	Human-in-the-Loop (HITL) Curation	19

3.2.1	ExtracTable	20
3.2.2	SciDaSynth	20
3.2.3	STRUCTSENSE	21
3.3	Summary of Research Gaps and Thesis Positioning	21
4	Approach	23
4.1	System Overview and Scalable Architecture	23
4.1.1	The Extraction Workflow	24
4.1.2	Designing for Universal Scalability	25
4.1.3	User Experience and Spatial Architecture	26
4.2	Intelligent Context Engineering	29
4.2.1	Source Context (The “Evidence Base”)	29
4.2.2	Structural Context (The “Neighborhood”)	30
4.2.3	Instructional Context (The “Task”)	32
4.2.4	Interactive Feedback Context (The “Steering Mechanism”)	32
4.3	Traceable Evidence and Search Strategy	33
4.3.1	Mitigating Hallucination via Evidence-Anchored Prompting	34
4.3.2	Enhancing Spatial Precision via Secondary Grounding	34
4.3.3	Balanced Algorithmic Search and the Dual-Layer Problem	34
4.4	AI as a Validation Tool	36
5	Implementation	38
5.1	Base Architecture and Technology Stack	38
5.1.1	Frontend Architecture	39
5.1.2	Backend Infrastructure	39
5.2	The Curation Workspace and User Interface	40
5.2.1	Document Ingestion and Workspace Entry	40
5.2.2	Split-Screen Workspace UI	40
5.2.3	The Questionnaire Toolbar	41
5.2.4	The AI Assist Interface	44
5.3	Implementing the Context Engine	44
5.3.1	Transforming the PDF: Semantic Chunks and Vector Retrieval	47
5.3.2	Structural Context Assembly	49
5.3.3	Implementing the HITL	49
5.4	Prompt Engineering and Data Validation	50
5.4.1	Designing the System Prompt	51
5.4.2	Dynamic User Prompt Assembly	52
5.4.3	Regex Extraction and Fail-Fast Validation	53
5.5	The Spatial Search Engine and Visual Rendering	54
5.5.1	The PDF Document Object Model and Multi-Stage Mapping	54

5.5.2	Execution of the Two-Pass Hybrid Search Algorithm	56
5.5.3	Geometric Bounding Boxes and Viewport Scaling	57
6	Evaluation	58
6.1	Architectural Requirements for the Evaluation Framework	58
6.2	Evolution of the Evaluation Framework	59
6.2.1	Phase 1: End-to-End Frontend Automation	59
6.2.2	Phase 2: Python-Based Logical Simulation	60
6.2.3	Phase 3: Headless TypeScript Client and API Parity	60
6.3	Automated Dataset Generation (The KG-Empire Dataset)	61
6.3.1	SPARQL Data Extraction	61
6.3.2	Robust PDF Acquisition	61
6.4	Multi-Metric Comparison Strategies	62
6.4.1	The Evaluation Challenge	62
6.4.2	Dynamic Metric Routing Engine	62
6.5	Resolving the Multiple Answers Discrepancy	63
6.5.1	The Challenge of Repeat Groups	64
6.5.2	The Three-Tier Transformation Strategy	64
6.6	Experimental Setup and Execution	65
6.6.1	Dataset Subsampling and Statistical Convergence	65
6.6.2	The Top-1 Suggestion Constraint	66
6.6.3	Thresholding for Binary Accuracy	66
6.6.4	Model Baselines and Configurations	66
6.7	Evaluation Results	67
6.7.1	Overall Performance	68
6.7.2	Granular Findings	68
6.7.3	Practical Utility (Top-3 Accuracy)	68
7	Discussion	70
7.1	The Impact of Contextual Injection	70
7.1.1	Algorithmic Overthinking	71
7.2	LLM Biases and Extraction Bottlenecks	71
7.2.1	Boolean Variables	72
7.2.2	Single-Select Classification	72
7.2.3	Multi-Select Arrays	72
7.2.4	Abstract Text Extraction	72
7.3	The Necessity of Human-in-the-Loop (HITL) Steerability	73
8	Conclusion and Future Work	75
8.1	Conclusion	75
8.2	Limitations	76

8.2.1	Exclusion of Multimodal Data	76
8.2.2	PDF Text Layer Artifacts and Highlighting Failures	76
8.2.3	The Absence of a “No Value” Pathway	77
8.2.4	Limitations of Automated LLM Evaluation	77
8.3	Future Work	77
8.3.1	Integration of Open-Source Local Models	77
8.3.2	Cross-Domain Generalization	78
8.3.3	Direct Platform Integration via APIs	78
8.3.4	Granular User Control and Multimodal Chunking	78
8.3.5	Live User Studies and Workflow Validation	78
A	Extended Interface Workflows	80
	Bibliography	83

List of Figures

4.1	The iterative Human-in-the-Loop extraction workflow.	24
4.2	Wireframe mockup of the split-screen user interface: a side-by-side architectural layout designed to synchronize the document viewer with the dynamic curation assistant.	27
4.3	The evidence-anchored extraction pipeline. Forcing the LLM to provide verifiable spatial coordinates optimizes search and allows for instant front-end rendering.	35
5.1	The document ingestion interface featuring drag-and-drop functionality and client-side validation.	41
5.2	The unified split-screen workspace co-locating the hierarchical questionnaire and the PDF renderer. The sample document utilized for this system demonstration is [9].	42
5.3	Global session controls: model, import, and validation.	43
5.4	AI Assist interface: menu and interaction modes.	45
5.5	The feedback and steering interface.	46
5.6	The History Manager for including or excluding past interactions.	46
5.7	Programmatic data funnel aggregating context streams into a single LLM prompt.	52
A.1	The evidence highlighting feature. Clicking an evidence snippet dynamically locates and highlights the exact source bounding boxes within the PDF viewer, enabling instant verification.	81
A.2	The bulk verification report interface. Triggered via a single click, this feature executes an automated LLM verification cycle across the entire questionnaire, outputting a consolidated quality assurance report of all finalized answers	82

List of Tables

5.1	Core fields of the interaction history schema.	50
6.1	Top-1 Suggestion Accuracy Aggregated by Question Type (N = 1,274) . .	68
6.2	Top-1 Suggestion Accuracy by Granular Question ID (N = 1,274)	69

Chapter 1

Introduction

The world is currently experiencing an explosion of scientific research. Traditionally, studies showed that the global volume of scientific output doubled approximately every nine years [10]. However, recent data indicates that this pace is increasing. According to the National Science Foundation, global publication output reached 3.3 million articles in 2022, based on data from the Scopus database of science and engineering (S&E) publications [40].

1.1 Motivation

Despite this massive growth, most of this information remains “locked” inside PDF files. Because the PDF format was fundamentally designed for visual rendering and print, it inherently lacks the internal semantic structure required for machine readability. Consequently, the data within these documents cannot be easily queried, aggregated, or compared by automated systems. This creates a significant barrier to evidence-based research. Evidence-based research is vital because it ensures that scientific decisions are grounded in the collective proof of many studies rather than a single, isolated observation. It allows researchers to filter out errors and identify the most effective solutions, providing a solid, verified foundation for every new discovery.

One of the most promising technological solutions to this data-silo problem is the Knowledge Graph (KG). Fundamentally, a Knowledge Graph is a paradigm designed to represent the real world by organizing information into a network of interconnected facts [18]. These graphs can model virtually any domain—from people and music to personal data and e-commerce—by representing distinct entities as nodes and the relationships between them as connecting edges.

In the context of academia, *Research* Knowledge Graphs apply this architecture to scientific literature, offering a vital solution to transition document-based stor-

age into structured knowledge [20]. By representing research components—such as methodologies, materials, or evaluation metrics—as distinct, interconnected nodes, unstructured text is transformed into machine-readable data. This allows researchers to query an entire network of knowledge simultaneously, making it possible to cross-examine the findings of thousands of papers in seconds.

A prominent implementation of this graph-based vision is the Open Research Knowledge Graph (ORKG) [20]. By representing research contributions as a semantic network of interlinked statements, scientific knowledge becomes intrinsically “Findable, Accessible, Interoperable, and Reusable” (FAIR) [52]. The ORKG operates as a dynamic, crowdsourced platform where literature can be collaboratively structured, queried, and compared. Once the data is properly modeled within the graph, researchers can utilize comparison tables that instantly highlight the precise differences between methodologies, evaluation metrics, and experimental setups across multiple publications.

To demonstrate the practical application of this approach, this thesis focuses on KG-EmpIRE [23], a specialized KG maintained within the ORKG ecosystem. KG-EmpIRE organizes empirical research in requirements engineering, currently comprising data extracted from 776 papers published in the annual International Requirements Engineering Conference (RE) between 1993 and 2025. Its primary goal is to provide a sustainable, up-to-date overview of the field by answering 77 specific “competency questions”—detailed queries regarding research methods, data analysis techniques, and validity threats. Crucially, the current subgraph is the result of entirely manual curation; domain experts painstakingly read and modeled the data by hand to answer 16 of the 77 proposed competency questions [24].

To remain useful and to meet the continuous demand for generating state-of-the-art comparisons of related work, KG-EmpIRE cannot be a static archive; it must be a living, evolving system. This requirement creates a continuous maintenance cycle that places an immense burden on human curators. First, the graph must be constantly expanded with new publications as they appear in the annual RE conference to prevent the dataset from becoming obsolete. Second, and perhaps more challenging, is the necessity for retroactive evolution: as the research community identifies new interests (e.g., deciding to track “Data Analysis Tools,” which was previously unrecorded), curators must manually revisit all 776 existing papers to update their content-data representations.

To understand why this continuous cycle is so burdensome, it is necessary to define what the “manual curation” process actually entails within the ORKG ecosystem. Curating literature into a structured graph is a rigorous, iterative loop consisting of three primary steps:

1. **Template Definition:** Curators must define an overall structural template

(a schema) that dictates exactly what data points will be extracted and modeled from the literature.

2. **Data Extraction:** Curators must manually read unstructured papers, locate the relevant information, and accurately fill out the template to create valid KG instances for each publication.
3. **Iterative Updating:** Curators must continuously integrate newly published papers into the existing graph. Furthermore, if a new scientific variable must be added to the template to capture emerging research trends, the process loops back to step one, requiring a retroactive extraction pass over all previously curated documents.

Despite the clear scientific benefits of maintaining a living KG, this strict manual curation workflow suffers from three critical limitations that prevent it from scaling to meet the needs of the modern research landscape:

- **Complexity and Expertise Requirements:** Scientific papers are not written in simple prose; they are filled with domain-specific jargon, complex tables, and nuanced methodologies. To curate a paper, a person must be an expert in specific domain. This high barrier to entry makes it impossible to “crowdsource” the task to non-experts.
- **Search Costs:** The most significant barrier to scaling a KG is the sheer amount of time required to locate information within a static document. It took an expert approximately 30 minutes on average to manually curate a single 10 page paper in KG-EmpIRE. When this time cost is extrapolated to thousands of papers, it becomes clear that manual maintenance is mathematically impossible to sustain.
- **The Coverage Gap:** Transitioning fluid prose text into rigid, machine-readable data (such as KG triples) is inherently an abstraction task. Because extracting complex narrative discussions into strict data templates naturally strips away nuance, and because manual curation is highly resource-intensive, KGs can practically capture only a fraction of a publication’s available information.

Hiring domain experts to perform manual data entry is not financially sustainable. If a single paper takes an expert 30 to 60 minutes to curate, processing a backlog of 1,000 papers represents roughly 500 to 1,000 man-hours. At academic or professional consulting rates, the cost of building a single knowledge graph can reach tens of thousands of euros. To democratize scientific knowledge, we must find a way to reduce these costs by an order of magnitude.

While it is tempting to leverage Large Language Models (LLMs) to fully automate this pipeline, deploying “black-box” AI in a rigorous scientific context introduces severe risks. LLMs are notoriously prone to hallucinations—generating plausible but factually incorrect statements or misinterpreting complex statistical data [43]. Furthermore, standard LLM outputs inherently lack provenance; they cannot reliably trace a generated value back to its exact origin within the source document. Because the core value of a scientific KG is absolute reliability, a dataset populated with even a marginal percentage of unverified or false information fundamentally loses its utility as an evidence-based tool.

This inherent conflict between scalability and data integrity necessitates a steerable Human-in-the-Loop (HITL) architecture. Rather than attempting to replace the curator, Artificial Intelligence must be integrated as an interactive assistant that accelerates the tedious search and extraction phases, while strictly preserving the human expert’s role as the final verification authority. This thesis addresses this motivation by introducing a platform designed to bridge the gap between AI-driven efficiency and human reliability. By providing automated, spatially verifiable evidence highlighting, the system drastically reduces the cognitive load and search time associated with manual curation, ensuring that high-quality scientific KGs can be constructed both rapidly and accurately.

1.2 Goal

The overarching goal of this thesis is to transform the scientific data curation process from a purely manual, document-centric task into a flexible, semi-automated workflow. Currently, curators are burdened with the dual responsibility of locating fragmented information within complex PDFs and interpreting that data to fit a strict structured template. (Note: While the process of designing and evolving these structured templates is itself a highly demanding task, building tool-support for schema creation falls outside the scope of this thesis and remains an important avenue for future work.) To alleviate the immediate extraction burden, this thesis introduces an intelligent assistant designed to actively support the curation process. Crucially, this system operates strictly on-demand and does not autonomously pre-process documents in the background. Because a curator can often spot straightforward information quickly, the interface allows them to manually enter and edit data at any time without AI intervention. The user only prompts the Large Language Model (LLM) for targeted suggestions. Consequently, the value of the proposed system extends beyond its AI integrations. It is architected to serve as a highly optimized, standalone curation environment. Even if a user chooses to bypass the AI entirely, the structured workflow and synchronized interface fundamentally streamline the traditional manual extraction process.

By establishing this robust HITL framework, the objective is to dramatically lower the high barrier to entry currently associated with data extraction. Rather than relying solely on highly specialized researchers, this tool is designed to empower a broader range of curators—even those outside the immediate domain—to successfully process scientific literature. Ultimately, this thesis aims to demonstrate that an on-demand AI assistant can act as a reliable force-multiplier, significantly reducing the cognitive burden of manual extraction while maintaining strict scientific quality standards.

To realize this overarching vision, the research is guided by three core sub-objectives:

1.2.1 Enabling Granular and Steerable Human-AI Interaction

A fundamental goal of this work is to transition away from opaque, “one-shot” AI extraction paradigms, which frequently overwhelm users with massive blocks of unverified data. Instead, this thesis aims to establish a highly granular interaction model designed to strictly limit the curator’s cognitive load. The objective is to provide targeted AI assistance while empowering the human expert to iteratively guide the model’s reasoning through natural language feedback and confidence-driven metrics. By doing so, the framework ensures that the human curator remains firmly in control of the extraction process, actively steering the AI rather than passively auditing it.

1.2.2 Establishing Verifiable Provenance to Ensure Data Trust

In scientific curation, an extracted data point is only as valuable as the evidence supporting it. A major barrier to adopting LLMs in high-stakes environments is their “black-box” nature and inherent risk of hallucination. Therefore, a critical objective of this research is to enforce strict data provenance. The thesis aims to develop a methodology that anchors AI-generated responses to their exact geometric origins within the source document. By providing transparent, spatially verifiable evidence, the goal is to transform manual, page-by-page validation into a rapid, high-confidence verification step, systematically mitigating the risk of fabricated data entering the KG.

1.2.3 Ensuring Cross-Domain Generalizability and Architectural Longevity

A recurring limitation of specialized information extraction tools is their tendency to become structurally overfitted to a single project schema or tightly coupled to a proprietary AI provider. To guarantee long-term utility, this thesis aims to design a highly adaptable, general-purpose architecture. The objective is twofold: first, to ensure schema independence, allowing the system to seamlessly process diverse data templates across distinctly different scientific domains; and second, to maintain strict model agnosticism. By decoupling the framework from any single LLM vendor, the research aims to ensure the tool can adapt to evolving KG structures and integrate future AI advancements without requiring fundamental software re-engineering.

1.3 Structure

This thesis begins with an overview of foundational concepts (Chapter 2) to establish the necessary background for the research. It introduces semantic knowledge infrastructures, focusing on Research Knowledge Graphs and the specific KG-EmpIRE methodology. Furthermore, it examines the application of LLMs in scientific extraction, highlighting their inherent limitations and establishing the foundational principles of HITL design.

The subsequent section examines related work (Chapter 3) to distinguish this thesis from existing approaches. It traces the evolution of knowledge extraction paradigms, moving from traditional rule-based systems to domain-specific pre-trained models, and finally to modern generative LLMs. By analyzing the current landscape of HITL curation, this chapter solidifies the research gaps and the specific positioning of this thesis.

The approach chapter (Chapter 4) details the conceptual design and scalable architecture of the proposed HITL framework. It outlines the strategy for intelligent context engineering—breaking down source, structural, instructional, and interactive context—and establishes the necessity of a verifiable search strategy to mitigate AI hallucination.

Following the conceptual design, the implementation chapter (Chapter 5) presents the technical execution of the system. It explains the core technology stack, the split-screen user interface, and the model-agnostic backend infrastructure. This chapter details the practical engineering required to build the Context Engine, construct dynamic prompts, and execute the spatial two-pass hybrid search algorithm for visual evidence rendering.

The core empirical focus of the work is addressed in the evaluation (Chapter 6).

This section details the methodology for testing the extraction pipeline, including the generation of the automated KG-EmpIRE dataset and the implementation of a dynamic metric routing engine. It explains how the framework resolves structural discrepancies, such as repeat groups, before presenting the quantitative results comparing a zero-shot baseline against the context-aware pipeline.

The discussion chapter (Chapter 7) then interprets these evaluation findings and examines their broader implications. It analyzes the direct impact of contextual injection on the model’s logical reasoning and breaks down specific extraction bottlenecks caused by inherent LLM biases, particularly in boolean and multi-select data types. Ultimately, this chapter utilizes the empirical data to justify the necessity of HITL steerability.

Finally, the work is summarized (Chapter 8), acknowledging the limitations of the current architecture and defining concrete directions for future research. This concluding section addresses challenges such as the exclusion of multimodal data and evaluation rigidity, while proposing future enhancements like the integration of open-source local models and cross-domain generalization.

Chapter 2

Background

To fully contextualize the design and architecture of the proposed extraction framework, this chapter establishes the foundational scientific and technological concepts upon which it is built. First, it explores the evolution of semantic data representation, transitioning from overarching Knowledge Graphs to specialized Research Knowledge Graphs (RKGs). This includes a detailed examination of the Open Research Knowledge Graph (ORKG), the KG-EmpIRE subgraph, and its dedicated exploration interface, EmpiRE-Compass. Following this structural foundation, the chapter examines the capabilities and inherent limitations of Large Language Models (LLMs) in processing dense scientific text. Finally, it explores the Human-in-the-Loop (HITL) paradigm, establishing why collaborative human-AI systems remain an architectural necessity for ensuring rigorous data provenance and mitigating algorithmic hallucinations in high-stakes academic curation.

2.1 Semantic Knowledge Infrastructures for Scientific Research

2.1.1 Knowledge Graphs

While the term Knowledge Graph (KG) was popularized by Google in 2012 to describe a shift in search engines from matching “strings to things,” it has since evolved into a fundamental architecture for artificial intelligence and data management. Academically, Hogan et al. (2021) [18] formally define a knowledge graph as “a graph of data intended to accumulate and convey knowledge of the real world, whose nodes represent entities of interest and whose edges represent potentially different relations between these entities.”

To understand how this works in practice, it is helpful to look at its underlying data structure. Unlike traditional relational databases that store data in

rigid rows and columns, a knowledge graph breaks information down into atomic, three-part statements known as semantic triples (often based on the Resource Description Framework, or RDF). Every piece of knowledge is logically structured as a continuous subject-predicate-object expression. Within this framework, the subject acts as the primary entity being described (representing a source node), the predicate defines the specific relationship or property connecting them (representing a directed edge), and the object represents either the related entity or a specific literal data value (representing the target node). By chaining millions of these simple triples together, the system builds a dynamic, web-like semantic network.

2.1.2 Research Knowledge Graphs

A Research Knowledge Graph is a specialized semantic network designed specifically to model the landscape of scholarly communication. Foundational literature establishes that an RKG presents an “opportunity to rethink the dominant paradigm of document-centric” communication—where the structure and semantics of findings remain hidden in text—and transform it into “knowledge-based information flows” through semantically rich, interlinked graphs [7]. Crucially, rather than merely containing bibliographic metadata (e.g., about articles, authors, and institutions), an RKG can capture semantic, machine-actionable descriptions of the actual scholarly knowledge [21].

The primary benefit of an RKG is its ability to render scientific knowledge machine-actionable, enabling automated querying, reasoning, and integration across studies. In doing so, RKGs align research outputs with the FAIR (Findable, Accessible, Interoperable, and Reusable) data principles introduced by Mark D. Wilkinson et al. [52]. By explicitly representing semantic relationships among research artifacts—such as the methodologies employed, datasets leveraged, and evaluation metrics reported—RKGs enable computational systems to autonomously aggregate, compare, and analyze findings across large-scale scholarly repositories. This structured representation facilitates automated reasoning, cross-study synthesis, and more efficient knowledge discovery.

The academic literature generally classifies RKGs into two distinct categories: generic and specific [47]. Generic RKGs operate at a massive, cross-disciplinary scale and focus almost exclusively on overarching bibliographic metadata and citation networks (e.g., authors, venues, DOIs). Conversely, specific RKGs are designed to open the “black box” of the publication. They focus on the deep semantic extraction of the actual scientific claims, methodologies, and empirical results embedded within the text of a particular academic domain or topic [47, 13].

2.1.3 ORKG

The ORKG is a fundamentally different kind of RKG, defined by its cross-domain and cross-topic architecture. Rather than remaining isolated, it serves as a federated infrastructure that hosts multiple academic domains under a single umbrella, providing a unified platform where researchers from any scientific discipline can organize deeply specific empirical data in a machine-actionable format [20].

A fundamental cornerstone of the ORKG’s architecture is its reliance on semantic templates, a feature detailed extensively in the ORKG handbook. The description of scientific findings is inherently complex and requires deep domain expertise to determine the appropriate granularity. To manage this complexity, ORKG templates empower domain experts to explicitly define the structure of research contributions, providing a guided data-entry framework for researchers who may not be experts in semantic data modeling. These templates are implemented using the Shapes Constraint Language (SHACL) [27]. By restricting inputs—such as ensuring a specific metric is always recorded as a standardized integer or float—these templates eradicate the inconsistencies that traditionally plague manual data recording. This strict adherence to standardized vocabularies ensures semantic interoperability, meaning computational systems can seamlessly understand and compare data drawn from disparate sources [20].

2.1.4 KG-EmpiRE

KG-EmpiRE is a domain-specific KG explicitly designed to systematically structure empirical research within the field of Requirements Engineering (RE) [24]. Rather than operating as an isolated database, KG-EmpiRE is hosted and maintained entirely within the ORKG infrastructure. Currently, the graph curates fine-grained scientific data extracted from hundreds of publications—specifically targeting the research track of the IEEE International Requirements Engineering Conference [23].

The primary motivation for developing KG-EmpiRE was to permanently resolve the sustainability crisis plaguing Systematic Literature Reviews (SLRs). Historically, researchers conducted extensive literature reviews to capture the evolving state of empirical RE, but these studies were fundamentally unsustainable. Because the extracted data was rarely made openly available within a unified system, subsequent researchers could not easily build upon or revise previous findings. Instead of collaboratively updating a shared knowledge base, the community was forced to constantly repeat labor-intensive extraction processes from scratch, resulting in massive academic redundancy and overlapping efforts [24, 15]. KG-EmpiRE addresses this by providing a community-maintainable infrastructure where empirical evidence can be continuously updated, queried, and built upon.

To guarantee that the data remains consistent and comparable across such a vast array of literature, KG-EmpIRE relies on a fixed, standardized data-entry template hosted within the ORKG. This template dictates the underlying graph structure for six core empirical themes: research paradigm, research design, research method, data collection, data analysis, and bibliographic metadata [23].

2.1.5 EmpiRE-Compass

While Research Knowledge Graphs (RKGs) like KG-EmpIRE successfully structure scientific data, they remain difficult for many researchers to access and use. Querying this data typically requires advanced technical skills, such as writing complex SPARQL queries or navigating intricate graph schemas. To overcome these usability barriers, EmpiRE-Compass was developed as an open-source, neuro-symbolic dashboard [25]. It serves as a user-friendly interface that allows researchers to easily explore, synthesize, and reuse structured literature data without needing specialized technical expertise.

The dashboard achieves this by unifying the structured, symbolic data of RKGs with the generative capabilities of LLMs [25]. Users can ask natural language competency questions, and the system dynamically generates interactive visualizations, interpretations, and semantic explanations directly from the underlying graph. Furthermore, EmpiRE-Compass is built with a highly modular architecture designed to extend beyond a single dataset. While initially developed for KG-EmpIRE, it has already been adapted to support a second distinct schema—the NLP4RE ID Cards [3]—demonstrating its flexibility as a general framework for sustainable scientific knowledge exploration.

2.2 Large Language Models in Scientific Knowledge Extraction

LLMs represent a significant breakthrough in artificial intelligence, fundamentally altering how machines process and generate human language. At their core, LLMs work through next-token prediction: given an input text, they predict the most likely next word based on patterns learned from massive amounts of training data [57]. Well-known examples include OpenAI’s GPT-4 [2], which achieves near human-level performance on many academic benchmarks, and open-source models like DeepSeek-V3 [33], which offer highly capable and cost-effective alternatives utilizing efficient architectures like Mixture-of-Experts. In the specific context of scientific knowledge extraction, these generative capabilities are transformative. Instead of relying purely on manual data entry, researchers can utilize LLMs to

automatically parse dense, unstructured academic PDFs, extract specific empirical variables, and map them directly into structured KGs.

2.2.1 Applications in Scholarly Research

Driven by their advanced semantic reasoning and text generation skills, LLMs are being applied across many scientific fields. In scholarly research, LLMs are increasingly utilized to automate literature analysis [53], generate scientific hypotheses [1], and extract structured data for systematic reviews [50]. While new applications continue to emerge because of these impressive results, LLMs still face severe inherent limitations [43].

2.2.2 Limitations and the Necessity of Human-in-the-Loop

The primary limitations hindering the reliable use of LLMs in rigorous scientific extraction are hallucinations and attention loss. Hallucination is a well-documented phenomenon where an LLM generates fluent and plausible, yet entirely fabricated or factually incorrect information that contradicts the source material or reality [43]. Furthermore, when processing extensive documents like full-length research PDFs, LLMs suffer from severe attention loss, commonly referred to as the “lost in the middle” phenomenon. As demonstrated by Liu et al. [34], an LLM’s retrieval performance degrades significantly when the relevant information is buried in the middle of a long input context, heavily favoring data located at the very beginning or end of the prompt .

To reduce these errors, developers use prompt and context engineering. For example, the Chain-of-Verification (CoVe) method by Dhuliawala et al. [14] forces the model to create fact-checking questions to verify its own draft, which significantly reduces hallucinations. Context engineering also helps by injecting relevant source text and targeted data to ground the model’s focus. However, while these strategies improve accuracy, they are not perfect. Building a reliable scientific KG still requires a HITL approach.

2.3 Human-in-the-Loop (HITL)

Human-in-the-Loop (HITL) machine learning is a collaborative paradigm that strategically integrates human intelligence into the algorithmic lifecycle to guide, validate, and refine computational outputs. In the modern era of LLMs, this concept has become even more critical. For example, Stephens et al. (2025) propose a “Scientist-in-the-Loop” architecture [46] specifically designed for semantic

KG construction, emphasizing that human experts must act as proactive semantic guides rather than merely serving as passive error correctors who fix LLM overgeneralization and ambiguity after the fact.

2.3.1 Advantages and Disadvantages

The primary advantage of the HITL approach is that it successfully merges algorithmic speed with human reliability. It leverages the rapid data-processing and pattern-recognition capabilities of AI while relying on human heuristic reasoning to navigate ambiguous edge cases, mitigate algorithmic bias, and intercept fabricated information (hallucinations). However, the primary disadvantage is that they require human labor, which is potentially resource-intensive and problematic to scale, creating a bottleneck that limits system scalability [6].

2.3.2 Application Areas in High-Stakes Domains

Because HITL provides an essential safety net, it is primarily deployed in sensitive, high-stakes domains where absolute trust, legal accountability, and strict accuracy are non-negotiable [49]. In medicine, for instance, AI models can rapidly process high-dimensional radiology scans to flag potential anomalies, but a human physician must stay in the loop to apply holistic patient context and make the final, legally binding diagnosis to ensure patient safety. Similarly, in autonomous driving, while AI handles routine navigation, humans serve as critical structural safety constraints capable of taking control during complex, unprecedented edge cases—such as extreme weather or unmapped construction zones—that fall outside the model’s training distribution. Furthermore, in the realm of scientific curation, extracting unstructured data for knowledge graphs requires absolute factual integrity. For example, by streamlining corpus creation and structured data extraction for knowledge graph integration, tools like ExtracTable [22] leverage LLMs to accelerate literature reviews; however, human validation remains essential to ensure data quality and prevent algorithmic errors from propagating into the scholarly record.

2.3.3 Suitability of HITL for the Current Research

The HITL paradigm is highly germane to the primary objective of this research: creating a reliable approach to accelerate scientific data extraction from PDF documents into a structured KG format. Because fully autonomous LLMs frequently struggle with attention degradation over lengthy documents and severe evidence hallucination, human intervention remains an architectural necessity to ensure data fidelity. By embedding a HITL approach into the extraction workflow, the system successfully mitigates the cognitive load on human curators while maximizing

extraction speed and accuracy. This collaborative synergy allows the computational system to perform the initial heavy lifting while the human expert resolves semantic ambiguities and validates spatial provenance. Ultimately, this ensures that the resulting structured outputs, destined for platforms like the ORKG, are strictly machine-actionable and maintain absolute scientific integrity.

Chapter 3

Related Work

Information extraction from scientific literature has undergone a massive paradigm shift, evolving from rigid syntactic pattern matching to dynamic, generative synthesis. To contextualize the architectural contributions of the proposed extraction system, it is necessary to examine the historical and contemporary progression of this field.

The evolution of automated scientific curation can be broadly categorized into three distinct eras. Early **Traditional Rule-Based and Machine Learning Systems** established the baseline for parsing formatted bibliographic metadata and identifying discrete keywords using heuristics and statistical algorithms, such as Support Vector Machines (SVMs) and Conditional Random Fields (CRFs). This foundational period was followed by the introduction of **Pre-Trained Language Models (PLMs)**, which utilized transformer-based architectures and contextualized word embeddings to shift the field from syntactic pattern matching to true semantic understanding and targeted span-extraction. Finally, the contemporary landscape is defined by **Large Language Models (LLMs) and Human-in-the-Loop (HITL) Frameworks**. These modern architectures leverage dynamic reasoning and Retrieval-Augmented Generation (RAG) capable of digesting massive context windows, increasingly integrating human experts to steer and verify AI-generated outputs. The remainder of this chapter will explore the technical evolution of these three paradigms in detail, examining how each architecture advanced the state-of-the-art before ultimately revealing the critical engineering bottlenecks that necessitate the proposed framework.

3.1 Evolution of Knowledge Extraction Paradigms

Before the rise of modern generative artificial intelligence, extracting information from scientific literature relied predominantly on rule-building and statistical

modeling. As the demand for deeper semantic extraction grew, the underlying architectures evolved.

3.1.1 Traditional Rule-Based and Statistical Machine Learning Systems

Before the advent of deep learning and generative AI, the automated processing of scientific literature relied predominantly on handcrafted rule-building and statistical modeling [39]. Historically, the field began by focusing on Metadata Extraction (ME)—identifying structured header and bibliographic elements using rule-based heuristics and statistical models like Conditional Random Fields (CRFs) [12, 5, 41]. While achieving high accuracy on structured metadata was a critical foundational step, the ultimate goal of scientific curation is extracting the actual unstructured knowledge embedded within the prose. Consequently, the core challenge of the field shifted toward Key-Insights Extraction (KIE).

Information and Key-Insights Extraction (KIE)

Moving beyond basic headers, researchers attempted to extract deep scientific knowledge. This extraction is broadly classified into sentence-level and phrase-level tasks. Sentence-level KIE attempts to categorize entire sentences into predefined structural roles (e.g., Background, Methods, Conclusion). Conversely, phrase-level KIE attempts to isolate specific semantic entities carrying potential information, such as the distinct “Problem,” “Domain,” or “Technique” utilized in a study [39].

Historically, sentence-level classification has been extensively explored using traditional machine learning algorithms such as Naïve Bayes, Hidden Markov Models (HMM), and Support Vector Machines (SVM) [45, 32, 17]. In contrast, phrase-level extraction has been comparatively limited by the scarcity of benchmark datasets, relying on rule-based heuristics and Conditional Random Fields (CRFs) applied to self-generated corpora [19, 28].

Limitations of Traditional KIE Paradigms

While these statistical algorithms established the baseline for automated knowledge extraction, a review of the literature reveals severe bottlenecks that render them unsuitable for modern, full-document semantic curation:

- **The Abstract-Only Bottleneck:** Because traditional statistical models require massive amounts of manually labeled training data, the vast majority of KIE studies from this era are restricted exclusively to document abstracts. As the number of target entities grows, the human effort required to annotate full-text scientific articles exponentially explodes [39].

- **Subjectivity and Private Datasets:** Phrase-level extraction in traditional systems suffers heavily from a lack of crisp annotation guidelines. The subjective nature of defining what constitutes a “Technique” or “Insight” leads to high individual bias. Furthermore, much of the reported success in this era was achieved on small, single-passage, or privately held datasets, often lacking rigorous cross-validation, clear methodological expression, or transparent dissemination of results [19, 26].

Ultimately, the domain rigidity, reliance on exhaustive manual feature engineering, and inability to efficiently scale to full-text PDF documents forced the academic community to abandon these early statistical methods in favor of the semantically aware architectures found in Pre-Trained Language Models.

3.1.2 Domain-Specific Pre-Trained Language Models (PLMs)

Pre-Trained Language Models (PLMs) are foundational neural network architectures trained on massive, unannotated text corpora using self-supervised learning objectives (such as masked language modeling). By learning to predict missing words or sentences, these models develop a deep internal representation of general syntax and semantics before being adapted to any specific task. To process the complex jargon and structural nuances of academic papers, the scientific community adapted this paradigm by training domain-specific PLMs directly on scientific literature.

By leveraging bidirectional transformer architectures, models such as SciBERT (trained on a massive multi-domain corpus of general scientific publications) [8] and BioBERT (designed specifically for biomedical literature) [29] generated deep, contextualized word embeddings explicitly tailored to academic prose. This extensive pre-training allowed researchers to efficiently fine-tune them for downstream extraction tasks like Named Entity Recognition (NER). Consequently, these domain-specific models gained the ability to read dense academic sentences and correctly highlight target entities, such as evaluation metrics, chemical compounds, or newly proposed methodologies.

Limitations of Pre-Trained Language Models

Despite their contextual superiority over statistical models, PLMs possess strict structural bottlenecks that hinder their application in full-document extraction tasks:

- **Input Length Restrictions (The 512-Token Limit):** PLMs are frequently constrained to a 512-token maximum, forcing full manuscripts to

be aggressively chunked. This mandatory splitting process risks severing long-range semantic dependencies, making it difficult to maintain broader context—such as successfully connecting a problem mentioned early in a document with a corresponding methodology located several paragraphs later [16].

- **Training Data Constraints and Scalability:** Specialized PLMs are strictly limited by their training data. For example, English-centric models like BioClinicalBERT [4] cannot process non-English texts without complete retraining or new architectures, severely restricting their universal scalability [16].

3.1.3 Generative Large Language Models (LLMs)

To circumvent the rigidity of span-extraction, the current paradigm utilizes Generative Large Language Models (e.g., GPT-4, Llama 3). These architectures transition extraction from a static classification task into dynamic, generative synthesis, capable of adapting to entirely novel scientific domains via zero-shot instruction orchestration.

In practice, scientific manuscripts are massive, multi-page documents with complex, highly unstructured visual layouts that can easily overwhelm standard context windows. To operationalize these models for scientific curation, the academic community moved beyond basic prompt engineering and developed advanced, fully automated pipelines that focus on two key architectural strategies.

First, instead of attempting to force an entire PDF into the LLM’s context window, modern frameworks prioritize a modular approach to isolate high-value, localized data. For example, the SciEx framework explicitly decouples the initial document parsing phase from the extraction phase [30]. It utilizes specialized toolkits to segment the visual PDF into distinct text, table, and figure elements, even employing Vision-Language Models (VLMs) to convert complex scientific charts into structured JSON representations. This data is stored in a contextualized database, allowing the system to utilize an iterative Retrieval-Extraction-Verification (REV) loop. By retrieving only the most semantically relevant text chunks and structured figure data for the primary LLM to process, the system dynamically targets localized evidence across multiple modalities without overwhelming the model’s context window.

Second, to force the LLM to output highly structured, machine-actionable data rather than conversational text, researchers are moving away from single-pass prompting toward advanced, multi-agent workflows. For example, to process lengthy and complex multi-omics research literature, researchers have deployed dual-LLM adversarial frameworks [31]. In this fully automated architecture, an

“Extractor LLM” generates an initial extraction based on a strict domain schema (capturing experimental data, analyses, and results). Subsequently, a “Verifier LLM” critically evaluates this output, providing iterative, adversarial feedback to detect formatting deviations, content omissions, and logical inconsistencies. This closed-loop interaction systematically forces the AI to self-correct, allowing the final structured data to converge toward a higher accuracy that empirically outperforms both manual expert annotation and traditional single-LLM baselines—all without requiring human intervention.

Limitations of Autonomous LLMs

While targeted retrieval and automated orchestration successfully scale the extraction process, deploying these pipelines as unsupervised, “black-box” agents introduces severe risks to scientific integrity. Without human oversight, these autonomous systems suffer from critical bottlenecks:

- **Algorithmic Confabulation (Hallucination):** The most critical vulnerability inherent to fully autonomous LLMs is the generation of fluent, highly plausible assertions that have no basis in the source text. As contemporary extraction studies note, generative models frequently “face challenges in consistency, factuality, and handling lengthy or multi-document contexts” [31]. If the model silently invents data—such as a sample size or metric—to satisfy a prompt’s structural demands, it permanently corrupts the downstream knowledge graph.
- **The Verification Bottleneck:** Current automated extraction approaches still exhibit a measurable gap to the ground truth [31, 50]. As a result, such systems cannot be deployed as fully autonomous curation agents; human curators are still required to manually verify extractions against the original source documents.

This manual verification requirement effectively negates the overarching efficiency gains of the automated system. It establishes a strict operational limit on fully autonomous pipelines and necessitates the field’s transition toward HITL architectures.

3.2 Human-in-the-Loop (HITL) Curation

Because fully autonomous LLMs suffer from unverified confabulations and spatial disconnects, their unsupervised deployment in academic curation is scientifically

unviable. To maintain absolute scientific integrity, the community relies on HITL architectures.

The integration of HITL workflows with generative AI has resulted in several specialized platforms aimed at curating scientific literature. A review of the current landscape reveals three prominent architectural approaches that demonstrate the evolving relationship between automated extraction and human oversight.

3.2.1 ExtracTable

Designed for downstream integration with ORKG, ExtracTable [22] automates the transformation of raw scientific corpora into structured knowledge graphs. The framework utilizes HITL interactions primarily during the initial setup phase to define and refine the extraction schema. However, once the overarching schema is established by the human curator, the system relies heavily on fully autonomous, zero-shot LLM prompting to actually extract the data points from the document. Consequently, ExtracTable strictly focuses on optimizing the initial corpus creation phase, prioritizing the speed at which a user can generate a baseline schema over providing granular, real-time human intervention during the actual text extraction.

3.2.2 SciDaSynth

SciDaSynth [51] is an interactive, LLM-powered system designed to extract and structure complex data from scientific literature into standardized tables. Utilizing a Retrieval-Augmented Generation (RAG) framework, it processes multimodal documents (text, tables, and figures), generates concise summaries to populate a vector database, and answers user queries by automatically inferring and filling a flat data structure. Recognizing that LLMs frequently struggle with domain context and cross-document consistencies, the system heavily integrates a HITL interface. Through a highly visual dashboard, the system flags uncertain or missing data and provides traceability directly to the source text, allowing users to rapidly verify facts.

Rather than conversationally steering the AI, the human-in-the-loop component in this system acts primarily as a verification and correction mechanism. The user does not write follow-up prompts to ask the AI to rethink its answers or look in different paragraphs. Instead, the AI performs the initial bulk extraction, and the researcher uses the visual interface to audit the AI’s work against the source documents and manually edit the final data table to ensure complete accuracy.

3.2.3 STRUCTSENSE

STRUCTSENSE [11] is an open-source, task-agnostic framework that simplifies complex data extraction by replacing a single, monolithic prompt with a collaborative, multi-agent architecture. The workload is divided among four specialized AI personas: an Extractor Agent that pulls the initial raw data, an Alignment Agent that prevents domain hallucinations by cross-referencing extracted terms against established scientific dictionaries using Ontology-Augmented RAG, a Judge Agent that evaluates extraction quality, and a Feedback Agent that manages human corrections. This architecture enables a highly intuitive human-in-the-loop mechanism. If an extraction error occurs, the human curator does not need to manually edit the output spreadsheet; they simply provide conversational, natural language instructions, such as asking the system to look closer at the methods section. The Feedback Agent processes this context and iteratively patches the specific mistake without restarting the entire extraction process.

3.3 Summary of Research Gaps and Thesis Positioning

An extensive review of the literature demonstrates a clear evolutionary trajectory from rigid statistical pattern matching to dynamic Generative Large Language Models. However, empirical evaluations of contemporary frameworks like SciDaSynth and STRUCTSENSE explicitly report that HITL workflows achieve significantly higher performance and extraction accuracy compared to fully automated baselines [51, 11]. These findings confirm that fully autonomous pipelines remain scientifically unviable without human oversight. Consequently, while the state-of-the-art has definitively shifted toward HITL architectures, critical engineering bottlenecks remain unresolved. Specifically, the literature reveals three persistent research gaps directly related to the problem statement of this thesis:

1. **Lack of a Complete, Curator-Centric Workflow:** Existing tools often function as disjointed AI experiments rather than comprehensive curation environments. They fail to provide a complete, UI-friendly agentic workflow that a curator can utilize seamlessly—not only as an AI-driven assistant but also as a dedicated, highly specialized manual extraction system when algorithmic intervention is unnecessary.
2. **Absence of Strong Evidence Grounding:** Even advanced multi-agent systems fail to adequately ground their outputs in verifiable physical evidence. There is a profound spatial disconnect; current frameworks output abstract data structures but fail to mathematically anchor that generated text to the

exact geometric coordinates (the physical bounding boxes) on the original PDF, leaving curators to manually hunt for the source of the data.

3. **Deficient Steerability and Contextual Control:** Existing systems broadly struggle to balance user control with AI memory. Current frameworks lack the ability for a curator to dynamically steer the AI using natural language while strictly maintaining and controlling the context of the ongoing extraction session. This deficit frequently leads to compounding logical errors and algorithmic hallucinations.

To directly resolve these three critical gaps, this thesis introduces a highly integrated, human-steerable extraction architecture within the EmpiRE-Compass platform. By proposing a unified, split-screen interactive workspace, the system provides a complete curation dashboard that gracefully supports both manual data entry and agentic AI assistance. To enforce strong evidence grounding, the framework employs a multi-stage spatial mapping pipeline and a custom hybrid search algorithm to render absolute-positioned bounding boxes directly on the document canvas. Finally, to achieve context-aware steerability, a dynamic Context Engine mathematically grounds the LLM using verified Parent and Sibling context, empowering the curator to guide the AI via natural language while preserving the strict relational hierarchy of the data. The subsequent chapters detail the systemic design, algorithmic implementation, and rigorous evaluation of this proposed architecture.

Chapter 4

Approach

As established in previous chapters, relying on zero-shot Large Language Models (LLMs) to automatically process full-length academic publications introduces severe risks regarding data hallucination and a lack of verifiable provenance. To mitigate these inherent limitations without sacrificing the generative power of modern AI, this thesis proposes a highly steerable, Human-in-the-Loop (HITL) extraction framework.

Unlike standard extraction pipelines that attempt to parse entire documents simultaneously, the proposed approach is built upon a granular, context-aware architecture. This design explicitly accounts for the hierarchical and interconnected nature of scientific curation templates (such as those used in KG-EmpIRE [23]). To operationalize these complex schemas, the system utilizes a questionnaire-based interface where the formal curation template has been transformed into a series of targeted inquiries. Because scientific data points are interconnected, the system does not extract them all at once; instead, it traverses this questionnaire in a sequence of smaller, logical tasks.

By dynamically engineering prompts grounded only in the most relevant document segments and previously user-validated answers, the system systematically controls the LLM’s cognitive scope. This chapter details the conceptual methodology and technical design of this framework, exploring the modular system architecture, the strategies for dynamic context engineering, and the spatial mapping algorithms required to visually anchor AI-generated answers directly to the source PDF.

4.1 System Overview and Scalable Architecture

The proposed system operates as an interactive, on-demand extraction assistant integrated directly into the curator’s workflow within the EmpiRE-Compass platform. Rather than functioning as a rigid, isolated tool, it acts as a specialized intelligent

layer designed to transition unstructured scientific literature into a structured Knowledge Graph.

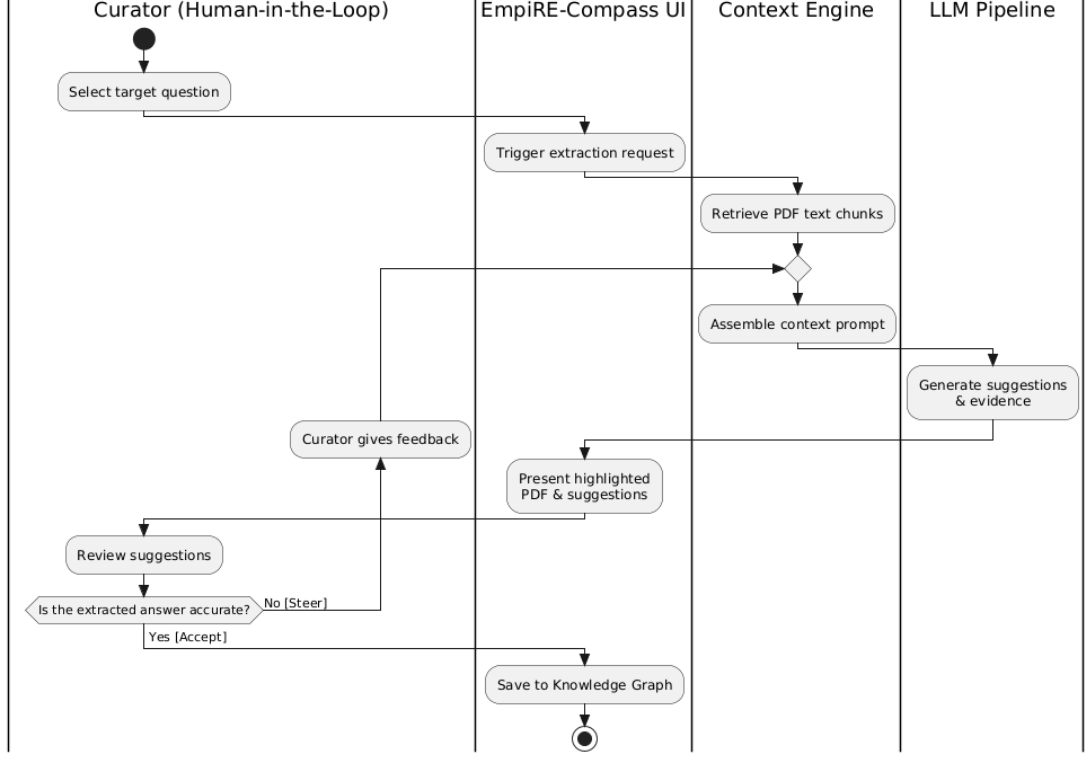


Figure 4.1: The iterative Human-in-the-Loop extraction workflow.

4.1.1 The Extraction Workflow

To establish a foundational understanding of the system’s architecture before detailing its specific components, it is necessary to trace the high-level lifecycle of a single data extraction request. The workflow, depicted in Figure 4.1, operates in five distinct phases:

1. **Initiation:** The cycle begins when the human curator interacts with a specific field in the questionnaire (representing the underlying graph schema), prompting the system for suggested answers.
2. **Context Retrieval and Assembly:** The system dynamically retrieves the most relevant sections of the source PDF. It then seamlessly combines this text with the deep structural context of the schema—capturing the complex

semantic rules and dependencies of the target data—alongside the curator’s prior feedback history.

3. **Generative Extraction:** This consolidated payload is dispatched to the LLM. Constrained by the provided context, the AI generates exactly three ranked suggestions, each strictly backed by verifiable, verbatim evidence from the source document.
4. **Spatial Verification:** The three suggested answers, along with their supporting evidence, are returned to the frontend architecture. The evidence is then programmatically mapped onto the PDF canvas, automatically highlighting the exact source text for the user.
5. **Human-in-the-Loop Steering:** The curator evaluates the suggested answers using the highlighted evidence. They can then make the final decision to apply the data, reject it, or provide natural language feedback to actively steer the AI toward a different perspective, thereby closing the iterative loop.

4.1.2 Designing for Universal Scalability

A primary architectural requirement for this system is long-term scalability. To ensure the platform remains robust, adaptable, and future-proof, the architecture was explicitly designed to decouple the extraction logic from both the underlying AI models and the specific curation schemas.

Model-Agnostic Foundation

The backend infrastructure is built upon a completely model-agnostic framework. This design choice guarantees that the system is not locked into a single proprietary AI provider. If API costs become prohibitive, or as more advanced open-source models enter the market, researchers can seamlessly switch to different LLMs without requiring custom integration efforts or overhauling the core application logic.

Schema-Independent Prompting

Beyond model flexibility, true scalability dictates that the system must easily adapt to entirely new domains of scientific research. This system employs a generic, dynamically constructed prompting architecture. The prompt does not inherently “know” what an empirical study is; instead, it programmatically reads the target data type (e.g., boolean, integer, multi-select) and the predefined options directly from the active schema template at runtime. Consequently, if the curation

questionnaire is radically altered, expanded, or replaced with a template from a completely different scientific discipline, the extraction system scales effortlessly without requiring a single line of backend code to be rewritten.

4.1.3 User Experience and Spatial Architecture

While the backend requires model-agnostic flexibility, a HITL system is fundamentally constrained by its frontend user interface; if the mechanism for verifying AI-generated data is overly complex or disorienting, the curator’s cognitive load increases, negating the efficiency gains of automation. Therefore, the physical layout and interactive design required an iterative architectural approach to successfully merge AI generation with manual curation.

From Black-Box to Assistant: Overcoming the Chatbot Paradigm

Initial design iterations explored a standard, conversational “chatbot” interface, a common paradigm for interacting with LLMs. In this approach, the user interacted with the questionnaire and the AI, but the source PDF was hidden or relegated to a separate application window.

However, this disjointed layout actively undermined the core philosophy of a steerable HITL system. When an LLM provided an evidence snippet, the curator was forced to constantly switch tabs or external windows to locate the broader context within the original research paper. This constant context-switching resulted in a fragmented user experience that felt like interacting with an opaque “black-box” rather than a transparent assistant. Furthermore, an effective extraction platform must not mandate AI reliance; if a curator opts to bypass the AI to manually read the text and extract an answer independently, the interface must seamlessly facilitate that traditional workflow. Relying on an isolated chatbot interface stripped the user of a unified workspace, making manual curation highly inefficient and disrupting their ability to track their overall curation journey. To prevent this broken experience and provide a comprehensive, one-stop environment for both automated assistance and manual record-keeping, it became evident that the source document and the questionnaire had to coexist within the exact same visual space.

The Overleaf-Inspired Split-Screen Layout

To resolve the spatial disconnect between the source material and the questionnaire, the system’s architecture was redesigned around a split-screen layout, taking direct inspiration from established academic writing tools such as Overleaf [42].

4.1. System Overview and Scalable Architecture

In this layout (illustrated in Figure 4.2), the interface is vertically divided. The left pane is dedicated entirely to rendering the original PDF manuscript, while the right pane houses the dynamic, hierarchical questionnaire and the AI interaction panels. This spatial architecture provides several critical advantages that directly enhance the efficiency and reliability of the curation process:

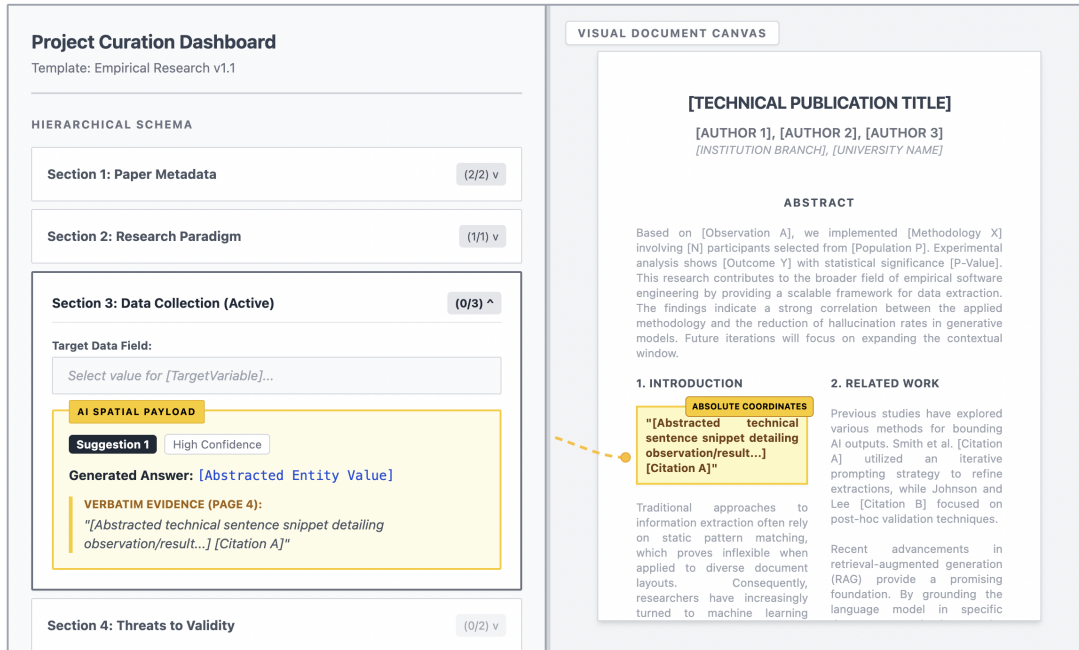


Figure 4.2: Wireframe mockup of the split-screen user interface: a side-by-side architectural layout designed to synchronize the document viewer with the dynamic curation assistant.

- **Continuous Grounding and Cognitive Relief:** The researcher never loses sight of the primary source material. By eliminating the need to continuously toggle between disjointed applications or browser tabs, the interface drastically reduces the user’s cognitive load. Every AI suggestion can be immediately cross-referenced against the visible document, effectively anchoring the AI’s abstract text to a physical, verifiable source.
- **Seamless Hybrid Workflow:** The unified workspace naturally accommodates both automated and manual curation methodologies. Because the complete document is perpetually accessible alongside the questionnaire, curators are not forced to rely on AI assistance. If a user opts to bypass the AI to manually read a specific section and extract an answer independently, they

can seamlessly transition between reading and data entry without breaking their established workflow or leaving the platform.

- **Accelerated Verification Cycle:** By placing the AI-generated suggestions immediately adjacent to the PDF rendering canvas, the spatial gap between a proposed answer and its source evidence is minimized. This proximity is crucial for the HITL methodology; when a suggestion is clicked, the system can automatically scroll the adjacent PDF to the exact page and highlight the relevant text, allowing the curator to visually validate the claim in a matter of seconds.

Cognitive Simplification and UI Consolidation

While the split-screen architecture solved the problem of document visibility, it introduced a new challenge: screen real estate. Because the extraction assistant is integrated directly into the broader platform rather than existing as a standalone application, it cannot rely on global sidebars or unbounded screen space. Early iterations of the split-screen design quickly became visually overwhelming, cluttered with multiple status bars, chat windows, and deeply nested question hierarchies.

To prevent cognitive overload, the interface underwent a rigorous “cognitive simplification” process, systematically consolidating features to maximize clarity:

- **Sequential Task Consolidation:** Early iterations separated the tracking of “missing data,” “invalid fields,” and “batch verification” into distinct UI panels. Recognizing that these tasks represent a strict, linear curation pipeline rather than isolated actions, the interface merges them into a single, unified progress tab. This centralization drastically reduces navigation overhead.
- **Fractional Progress Indicators:** To provide clear status visibility without adding visual clutter to the unified tracking system, the interface employs clean, fractional indicators (e.g., “0/3”). Placed at the end of each questionnaire section, these allow the curator to assess their exact position in the completion pipeline at a single glance.
- **Vertical Compression via Accordions:** As schema requirements grow, data fields become extensively long. To prevent endless scrolling and manage vertical length, the interface implements an auto-closing accordion architecture. Expanding a new section automatically collapses the previous one, strictly limiting the vertical scroll space and keeping the user focused solely on the active context.
- **Color-Depth Hierarchy:** Visualizing deeply nested parent-child relationships using traditional tree structures with connecting arrows created severe

visual fatigue and UI clutter. Inspired by the ORKG platform, the system abandons structural lines in favor of a color-depth visualization. Deeper hierarchical levels are represented by progressively darker background colors, providing an intuitive, space-efficient visual cue of the user’s current depth.

- **Minimalist Signifiers:** To further reduce visual clutter, explicit text instructions were replaced with minimalist UI signifiers. Required questions are denoted by a *, complex instructions are hidden behind info (i) tooltips, and all AI-specific actions were condensed into a single centralized dropdown menu.

4.2 Intelligent Context Engineering

A fundamental challenge in extracting structured data from scientific literature is that isolated questions often lack sufficient meaning, and feeding an entire PDF to an LLM introduces severe operational bottlenecks. When presented with a full paper, LLMs struggle with token limits and attention degradation, particularly for information buried in the middle of the document [34]. Furthermore, scientific data is fundamentally hierarchical, and the questions within a curation template are deeply interconnected. If an LLM is only provided with a single question and a massive wall of text, it may misinterpret the query or generate an answer that is technically correct but semantically wrong for the specific data schema. To solve this, the proposed approach employs a highly structured context engineering strategy. This strategy systematically constructs a comprehensive prompt that provides the LLM with distinct layers of contextual awareness, ensuring the model’s output is accurate and relevant.

4.2.1 Source Context (The “Evidence Base”)

While the system’s underlying architecture handles the dynamic retrieval of relevant PDF sections, the Source Context represents how this targeted information is actively injected into the LLM’s prompt. Merely asking an LLM a question relies on its internal, pre-trained knowledge, which is highly susceptible to hallucination in specialized scientific domains. By explicitly prepending the dynamically retrieved text chunks to the query, the system grounds the model in a strict factual boundary. The core philosophy here is that while removing useless tokens from the prompt is essential for efficiency, adding the right useful context is equally critical. This foundational layer of context forces the AI to act as a targeted reading comprehension engine rather than a creative text generator, ensuring its suggestions are strictly derived from the provided text.

The Evolution of Context Retrieval: Overcoming Static Mapping

A critical component of building this Source Context lies in how the system manages the vast amount of text within a scientific PDF. Initially, the approach attempted to mitigate token limits and attention degradation by hardcoding specific questions to predetermined sections of the research paper (e.g., mapping methodology questions strictly to the “Methods” section). However, this static mapping introduced severe scalability and accuracy limitations:

- There was no absolute guarantee that a question’s answer would be confined exclusively to its designated section.
- If a mapped section was enormously large, the system still encountered restrictive token limits.
- This approach was fundamentally unscalable across different domains; every time a new questionnaire or schema was created, the specific mapping logic would have to be manually redefined.

Dynamic Semantic Mapping

To create a truly intelligent and scalable framework for the Source Context, the architecture required a dynamic solution for retrieval. Rather than relying on rigid section mapping, the approach transitioned to a semantic search strategy managed by the Context Engine.

In this updated architecture, the text of the PDF is divided into manageable chunks, and vector embeddings are generated for each chunk. When a curator initiates an extraction request, the system simultaneously generates an embedding for the specific target question and calculates the cosine similarity between the question and the document chunks.

This dynamic retrieval architecture provides two major advantages. First, it efficiently filters out irrelevant information, ensuring that the LLM only processes the most pertinent text for its Source Context. This approach significantly mitigates concerns regarding token limitations and the “lost-in-the-middle” attention degradation. Second, it establishes a universally scalable approach. Because the retrieval is based on semantic meaning rather than hardcoded rules, the system can seamlessly process completely new questionnaires and schemas without any prior manual configuration.

4.2.2 Structural Context (The “Neighborhood”)

Because scientific data is inherently hierarchical, an isolated query such as “What was the sample size?” is fundamentally ambiguous. Without knowing the es-

established parameters of the specific experiment being evaluated, an LLM might erroneously extract the sample size of a preliminary pilot study rather than the main evaluation. To resolve this ambiguity, the system dynamically injects Structural Context, creating a semantic “neighborhood” around the target question. Crucially, this context does not merely supply the surrounding schema structure; it dynamically feeds the LLM the previously curated and validated answers from adjacent fields. By leveraging this accumulated state, the system grounds the AI’s reasoning in the specific reality of the curator’s ongoing session through three distinct mechanisms: Parent Context, Sibling Context, and Adjacent Entry Context.

Parent Context (The Vertical Hierarchy)

Parent Context explicitly passes the validated values of higher-level questions down to their nested sub-questions. In complex empirical schemas, lower-level data points are strictly dependent on their parent nodes. For example, a target question might ask, “What is the stated sub-question in the study?”. In isolation, an LLM might erroneously extract any minor question posed anywhere in the text, such as a rhetorical question in the introduction or an unresolved question in the conclusion.

However, if the curator has already validated the answer to the parent question (“What is the stated research question in your study?”) with a specific value—such as “How do agile practices affect technical debt?”—injecting this Parent Context explicitly constrains the AI. The LLM is forced to anchor its search around that primary research objective, specifically looking for the methodological or empirical sub-questions that logically decompose that exact parent question. By passing the previously validated state down the hierarchy, the system ensures vertical consistency and prevents the extraction of out-of-scope data.

Sibling Context (The Horizontal Boundary)

While Parent Context guides the AI vertically, Sibling Context provides critical horizontal constraints by injecting the validated answers of adjacent questions located at the exact same hierarchical level. For instance, if the AI is tasked with answering, “Is the research question explicitly highlighted in the text?”, it is immensely beneficial to provide the Sibling Context of the question itself (e.g., if the sibling field “What is the stated research question?” has already been answered by the curator with “How do agile practices affect technical debt?”). By feeding the LLM the exact string of the already-extracted research question, the AI no longer has to infer the subject matter. It can directly locate that specific concept within the PDF and strictly evaluate its presentation formatting. This allows the model to leverage the curator’s prior verification work to achieve much higher precision,

drastically reducing the cognitive burden of subsequent extractions.

Adjacent Entry Context (The Sequential Boundary)

Finally, while Parent and Sibling contexts define relationships between different schema nodes, Adjacent Entry Context manages the internal, iterative structure of a single, multi-valued node. Many studies feature complex experimental setups where a single question may have multiple valid answers, making the extraction process highly iterative. To handle this interconnected nature of the data, the system utilizes Adjacent Entry Context. When a question requires more than one answer, any previously written answer is fed back to the LLM. The model is explicitly instructed that this specific answer is already documented, prompting it to look for new directions or alternative perspectives. This dynamic, continuously updating context is crucial for preventing the AI from fixating on a single, dominant data point within the text. Instead of repeatedly generating the most obvious recognized answer, the system actively forces the LLM to broaden its search space and uncover distinct, previously unextracted information that satisfies the multi-value requirement.

4.2.3 Instructional Context (The “Task”)

While structural context guides the meaning of an extraction, Instructional Context enforces the required data format. To guarantee compliance with the rigid semantic templates driving the curation process, the system translates the questionnaire’s structural constraints directly into the AI prompt. It defines the precise extraction task by providing the target question alongside its mandatory data type (e.g., Boolean, Integer, or String). For categorical fields, the prompt dynamically includes the predefined array of allowable options. By explicitly bounding the generative output to the active schema, this context ensures the AI’s response is perfectly formatted and immediately machine-actionable.

4.2.4 Interactive Feedback Context (The “Steering Mechanism”)

In a robust HITL system, the curator’s role deliberately shifts from manual data entry to data verification and active guidance. For this paradigm to succeed, providing a fluid mechanism for the user to steer the AI is the most critical component.

Evolution of the Feedback Mechanism

Initially, the system attempted to capture user corrections using a structured, criteria-based evaluation checklist. However, during initial prototyping, this approach proved to be overly mechanical and rigid for practical use. Consequently, the interface was redesigned to facilitate natural language feedback, paired with intuitive “like” and “dislike” evaluations. The underlying logic is highly pragmatic: if the LLM’s suggestion is generally on the right track, the curator can gently push it further toward the exact answer; if it is entirely wrong, they can instruct the model to abandon its current logic and explore a completely new perspective.

Chat-Mimicry and Granular Context Control

To make this steering process iterative, the system stores all prior user feedback. When a user prompts the AI again, this history is dynamically injected into the prompt as Interactive Feedback Context, effectively mimicking the continuous, evolving nature of a chat interface. However, standard chat interfaces suffer from a major architectural flaw: they force the AI to remember everything, which can compound errors over time.

To solve this, the proposed workflow moves one step beyond a standard chat interface by granting the user granular control over the AI’s contextual memory. The curator can explicitly select which specific pieces of past context to send to the model and which to omit. This design acts as a critical fail-safe. If the LLM begins to hallucinate based on a previous prompt, or if the user realizes a piece of their own past feedback was flawed, they can simply remove that specific interaction from the context window. This ensures the AI’s reasoning remains fully controllable by the user and prevents past mistakes from corrupting future extractions.

4.3 Traceable Evidence and Search Strategy

A critical limitation of applying LLMs to document curation is that while they excel at generating semantic answers, they inherently lack spatial awareness. An LLM can provide a relevant sentence, but it cannot organically connect that generated text back to its exact physical location within the source PDF. In a scientific context, an answer without a verifiable source is unusable. Furthermore, simply taking the AI’s generated text and running a standard string search across the entire document is computationally inefficient and drastically increases the time a user spends waiting for the system to respond. To solve this, the proposed approach establishes a Traceable Evidence Pipeline that forces the AI to actively prove the origin of its claims.

4.3.1 Mitigating Hallucination via Evidence-Anchored Prompting

To prevent the model from fabricating answers, the system employs an evidence-anchored prompt engineering technique. Instead of merely asking the LLM to extract an answer, the prompt strictly requires the model to return the exact verbatim sentence from the text, alongside the specific page number where it found that information.

This dual-requirement serves two critical purposes. First, it drastically reduces the search space for the underlying algorithm; knowing the exact page makes the visual highlighting process highly efficient. Second, and more importantly, it acts as a powerful anti-hallucination mechanism. By forcing the LLM to anchor its conceptual answer to a tangible spatial coordinate (the page number), the risk of evidence hallucination is substantially reduced. If the model cannot formulate a verifiable path back to the source text, it is constrained from generating a fabricated response. This cyclical extraction and spatial grounding process is illustrated in Figure 4.3.

4.3.2 Enhancing Spatial Precision via Secondary Grounding

While the initial evidence-anchored implementation successfully reduced fabricated evidence, practical application revealed a secondary issue: the LLM’s spatial reasoning was still imprecise. The model would extract the correct verbatim text, but the page numbers it provided were frequently incorrect. When the AI provided a wrong page number, the underlying search algorithm was forced into a “worst-case scenario,” reverting to scanning the entire document and causing unacceptable delays for the user.

To resolve this without sacrificing the speed of the search algorithm, the approach applied the spatial grounding logic one step further. The prompt was iteratively refined to ask the LLM to provide not only the page number but also the line number of the extracted evidence. While the system does not strictly need the line number for its internal search logic, imposing this extreme level of required precision forces the LLM to pay much closer attention to the spatial layout of its context window. As a result of this secondary spatial forcing, the accuracy of the model’s page number predictions increased dramatically.

4.3.3 Balanced Algorithmic Search and the Dual-Layer Problem

Even with the LLM reliably outputting both the exact text snippet and the correct spatial coordinates (the page number), physically highlighting that text within the

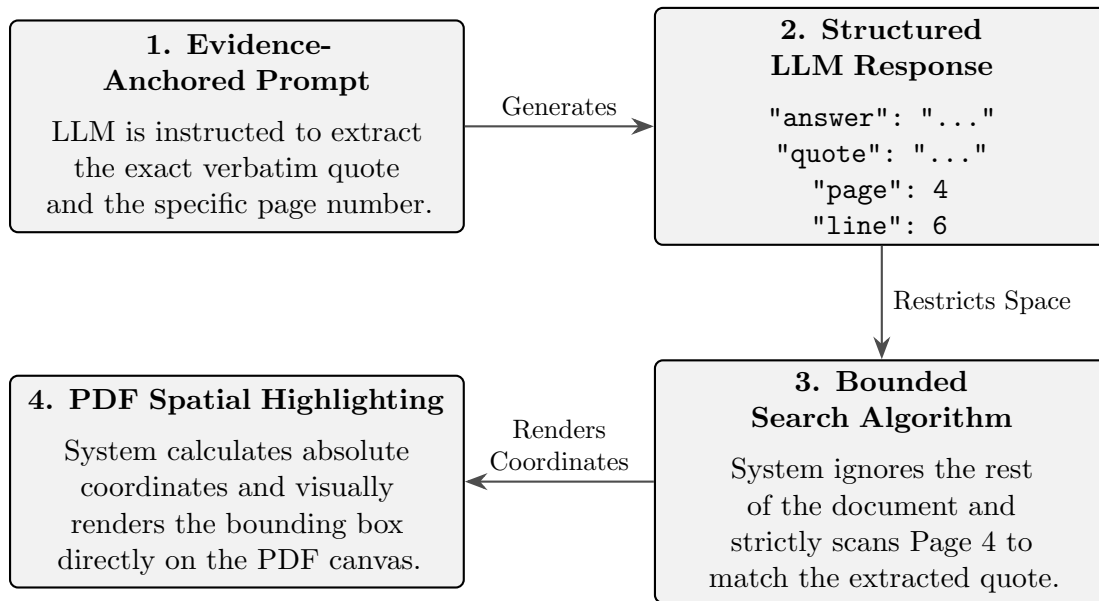


Figure 4.3: The evidence-anchored extraction pipeline. Forcing the LLM to provide verifiable spatial coordinates optimizes search and allows for instant frontend rendering.

user interface presents a complex algorithmic challenge. An LLM generates a clean, continuous string of text, but it cannot natively connect that sentence directly to the PDF’s internal structure. To achieve functional highlighting, the system must seamlessly combine the AI-generated output layer with the PDF rendering layer.

The Dual-Layer Text Discrepancy:

The fundamental issue with searching within PDFs is that they consist of two distinct layers: the visual image layer that the user sees, and the hidden text layer embedded behind the image. Frequently, the hidden text layer contains optical character recognition (OCR) errors or formatting artifacts. For example, the visual layer might clearly display the word “will”, but the underlying text layer might encode it as a typo, such as “wilf”.

When the system extracts this flawed text chunk and feeds it to the LLM as context, the LLM’s natural language processing capabilities usually automatically correct the typo in its generated response. While this results in a grammatically perfect answer, it fundamentally breaks standard exact-match search algorithms; searching the PDF for the corrected word “will” fails because the document strictly contains the typo “wilf”.

The Efficiency vs. Accuracy Trade-off:

To resolve this discrepancy, the system’s underlying search logic required extensive research and trial-and-error. The development process involved testing seven different string-matching algorithms to find the provided LLM text within the PDF. This testing revealed a strict computational trade-off: algorithms capable of high accuracy (such as complex fuzzy-matching algorithms that can gracefully handle typos like “wilf”) tend to be highly inefficient computationally. Applying these heavy algorithms blindly results in unacceptable UI blocking, leaving the user stuck waiting on the page while the system searches. Conversely, highly efficient algorithms are too rigid and fail whenever the LLM auto-corrects a PDF formatting quirk.

The Hybrid Search Solution:

To ensure the system remained as efficient as possible with the least amount of downtime, the final approach utilizes a custom-developed search algorithm that represents the “best of both worlds”. By engineering an algorithm that strikes an optimal balance between the high accuracy needed for fuzzy matching and the low time-complexity needed for a responsive UI, the system prevents performance bottlenecks.

Furthermore, this balanced algorithm works in tandem with the spatial forcing mechanisms described previously. Because the LLM accurately provides the target page number, the search algorithm avoids the computationally expensive “worst-case scenario” of scanning the entire document. By strictly limiting the balanced search logic to the verified page, the system can instantly map the AI’s string to the PDF’s coordinates. The result is a seamless user experience: as soon as the curator clicks on the AI’s provided evidence, the system automatically scrolls to the exact page and instantly highlights the relevant text, bridging the gap between abstract AI reasoning and physical document structure.

4.4 AI as a Validation Tool

Finally, this steerable workflow is not exclusively limited to generating new data suggestions; it is designed to seamlessly support and validate manual curation efforts. In scenarios where a curator prefers to extract data independently, they can utilize the AI as a passive verifier. By feeding their manually entered answer to the LLM, the curator can request the model to cross-reference their input against the source text and provide an objective confidence assessment.

To further streamline the review process, the system scales this capability from individual fields to the entire document through a dedicated Batch Verification

pipeline. Once a curator finishes populating the schema, they are not forced to verify each entry individually. Instead, they can initiate an automated, macro-level review. The system systematically evaluates all entered answers against the source PDF in the background, ultimately generating a comprehensive validation report. By offering both on-demand granular verification and automated batch auditing, the system ensures the AI adapts entirely to the curator’s workflow, acting as both an active extraction assistant and a rigorous quality assurance tool.

Chapter 5

Implementation

This chapter translates the conceptual HITL architecture presented in Chapter 4 into a functional engineering reality. This extraction framework was engineered as a specialized data-contribution module integrated directly into the EmpiRE-Compass ecosystem¹.

As established in previous chapters, EmpiRE-Compass serves as a central dashboard for exploring and synthesizing structured empirical research. It was deliberately selected as the target environment for this implementation because of its rapidly expanding scope. As the platform continuously incorporates increasingly complex semantic templates—such as the recent integration of NLP4RE ID Cards—curators require a robust mechanism to actively extract and contribute new data. By deploying a steerable, questionnaire-based extraction assistant directly into this existing dashboard, the system effectively bridges the gap between knowledge consumption and knowledge creation. This ensures that domain experts can seamlessly transition unstructured PDFs into structured graph data without leaving their established exploratory workspace.

Consequently, the technical details outlined in this chapter focus on establishing a highly scalable architecture. It details the specific libraries, APIs, and custom algorithms utilized to process complex academic PDFs, dynamically generate evidence-based suggestions based on varying schema constraints, and render the seamless split-screen interface required for effective human validation.

5.1 Base Architecture and Technology Stack

The selection of the technology stack was driven by four primary architectural requirements: type safety, responsive asynchronous performance, model-agnosticism,

¹The full integration codebase is available via the [masterarbeit branch](#).

and privacy-conscious data processing. To satisfy these constraints, the implementation follows a decoupled client-server architecture unified by a **TypeScript** ecosystem [36]. By leveraging a modern React-based client alongside a robust Node.js and Express server infrastructure, the architecture ensures seamless, end-to-end data validation and allows the system to handle complex scientific schemas without structural degradation during transmission.

5.1.1 Frontend Architecture

The extraction module is engineered as a two-stage Single Page Application (SPA). To support high-density information requirements and maintain interface responsiveness while rendering heavy PDF documents, the following foundational technologies were utilized:

- **React Framework:** Forms the core UI architecture. The system leverages React’s component-based rendering to handle dynamic UI elements (e.g., active highlights, split-screen layouts). By managing these elements through localized component state, the interface ensures high frame rates during rapid Human-in-the-Loop interactions [35].
- **Material UI (MUI):** A widely-adopted React component library that implements Google’s Material Design system. It provides a comprehensive suite of pre-built, accessible, and high-quality UI components that ensure a professional and consistent user experience [38].
- **React-PDF & PDF.js:** A robust, industry-standard library developed by Mozilla for parsing and rendering Portable Document Format (PDF) files. It utilizes a web-worker-based architecture to provide high-performance, canvas-based document visualization directly within the browser environment [37].
- **Redux Toolkit:** Utilized strictly as a centralized, read-only store for global configuration state (such as the selected LLM provider settings). This provides a single source of truth for static settings without causing unnecessary UI lag [44].

5.1.2 Backend Infrastructure

The backend operates on a Node.js runtime environment utilizing the Express web framework. The server acts as a secure orchestration layer, designed to execute model-agnostic queries and perform local vectorization without introducing abstraction-heavy overhead. This infrastructure is powered by the following components:

- **Vercel AI SDK:** Rather than introducing a redundant AI framework from scratch, the extraction module hooks directly into the Vercel AI SDK that was already established within the broader EmpiRE-Compass platform. Leveraging this pre-existing infrastructure provides a unified translation layer, allowing the backend to stream responses from various Large Language Models (e.g., OpenAI, Mistral) and achieve true model-agnosticism while maintaining complete architectural consistency [48].
- **Transformers.js:** Facilitates the local execution of the `all-MiniLM-L6-v2` quantized embedding model directly within the Node.js environment, enabling efficient, on-device text vectorization [55, 54].

5.2 The Curation Workspace and User Interface

This section details the functional implementation of the extraction workspace, moving from the initial document ingestion to the interactive split-screen environment where AI assistance and human verification converge.

5.2.1 Document Ingestion and Workspace Entry

The workflow begins at a dedicated entry point designed for secure document handling. The interface provides a drag-and-drop zone (Figure 5.1) that executes immediate client-side validation. This ensures that only properly formatted PDF manuscripts are passed into the extraction pipeline, preventing backend errors during the subsequent text-parsing and vectorization phases.

Upon successful validation, the system must transition the user to the core curation workspace without degrading browser performance, the frontend immediately generates a lightweight Binary Large Object (Blob) URL for the validated file. By passing this temporary URI directly through the client-side router state, the system prevents memory bloat and ensures near-instantaneous document rendering in the next view while maintaining a minimal application footprint.

5.2.2 Split-Screen Workspace UI

Upon successful upload, the system initializes the core workspace (Figure 5.2). This interface is architected as a vertically divided split-screen to eliminate the cognitive overhead of switching between a source document and a data entry form.

- **Integrated PDF Viewer (Right Pane):** The viewer provides essential reading affordances, including zoom controls, a page-jump indicator, and

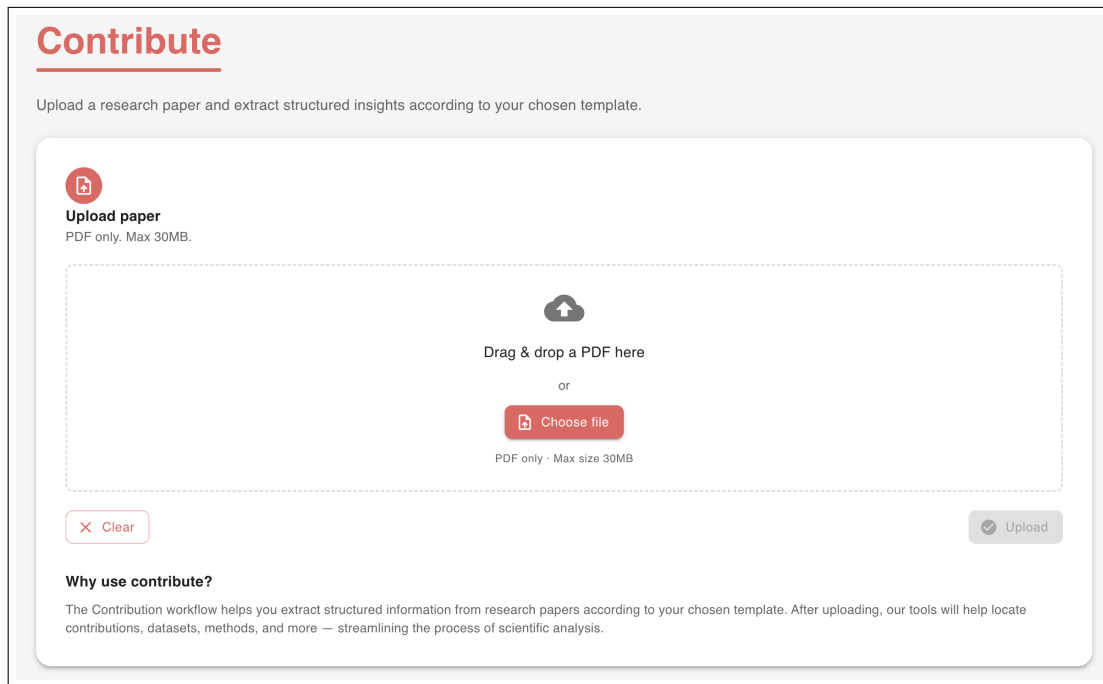


Figure 5.1: The document ingestion interface featuring drag-and-drop functionality and client-side validation.

manual navigation. It serves as the “visual ground truth” where all AI-generated evidence is eventually rendered as spatial highlights.

- **Dynamic Questionnaire (Left Pane):** The left pane renders the hierarchical questionnaire as an interactive form. Each field within this pane is linked to the AI engine, allowing the curator to trigger localized extraction or verification requests.

5.2.3 The Questionnaire Toolbar

Positioned at the apex of the questionnaire, a compact toolbar (Figure 5.2) centralizes the high-level orchestration of the curation session. This toolbar exposes four critical workflow modules:

1. **AI Selector:** The AI Selector (Figure 5.3a) allows the curator to toggle between various Large Language Model providers (e.g., OpenAI, Groq, Mistral). This ensures the architecture remains model-agnostic, allowing for real-time adjustments based on the required reasoning depth or API availability.

Empirical Research Practice
v1.1
Configure AI settings to use OpenAI or Groq models
PDF Status: Text extraction completed

Context 0/2

Research Paradigm 1/1

Research Questions & Answers 0/2

Data Collection 0/2

Data Analysis 0/5

Threats to Validity 0/1

Software Requirements As Negotiated Win Conditions
Barry Boehm, Prasanta Bose, Ellis Horowitz, and Ming-June Lee
USC Center for Software Engineering
Department of Computer Science
University of Southern California
Los Angeles, California
e-mail: (boehm, pbose, horowitz, milee)@pollux.usc.edu

Abstract
Current processes and support systems for software requirements determination and analysis often neglect critical needs of important classes of stakeholders and limit themselves to concerns of the developers, users and customers. Besides developers, customers, and users, these stakeholders can include maintainers, interfaces, testers, product line managers, and sometimes members of the general public.
This paper describes the results to date in researching and prototyping a Next Generation Process Model (NGPM) and support system (NGPS) which directly addresses these issues. The NGPM emphasizes collaborative processes, involving all of the significant constituents with a stake in the software product. Its conceptual basis is a set of Theory W (win-win) extensions to the Spiral Model of software development.

1. Introduction
1.1 Context and Motivation
Current processes and support systems for software requirements often reinforce a sequential consideration of the concerns of the various stakeholders in the software process and its resulting products. In the most frequent sequence, fairly vague expressions of user's information processing needs are translated by system analysts into a set of product specifications (sometimes very precise, sometimes fairly general, but often inaccurate in important respects).
These specifications are used by a customer to define a contract with a developer generally with minimal user participation. Even if the first product of this contract is an initial increment of an evolutionary development, the detailed concerns of the product's users, maintainers,

Also, this sequential participation in stages of the software process leads to win-lose situations among the various stakeholders. These may result from coalitions of some parties at the expense of absent parties; from an "out of sight, out of mind" phenomenon; or from plain ignorance of absent parties' likely win conditions.

Table 1. Frequent Software Development Win-Lose Patterns (which generally evolve into lose-lose situations).

Proposed Solution	"Winners"	Loser
Quick, cheap, sloppy product	Developer, Customer	User
Lots of "bells and whistles"	Developer, User	Customer
Driving too hard a bargain	Customer, User	Developer

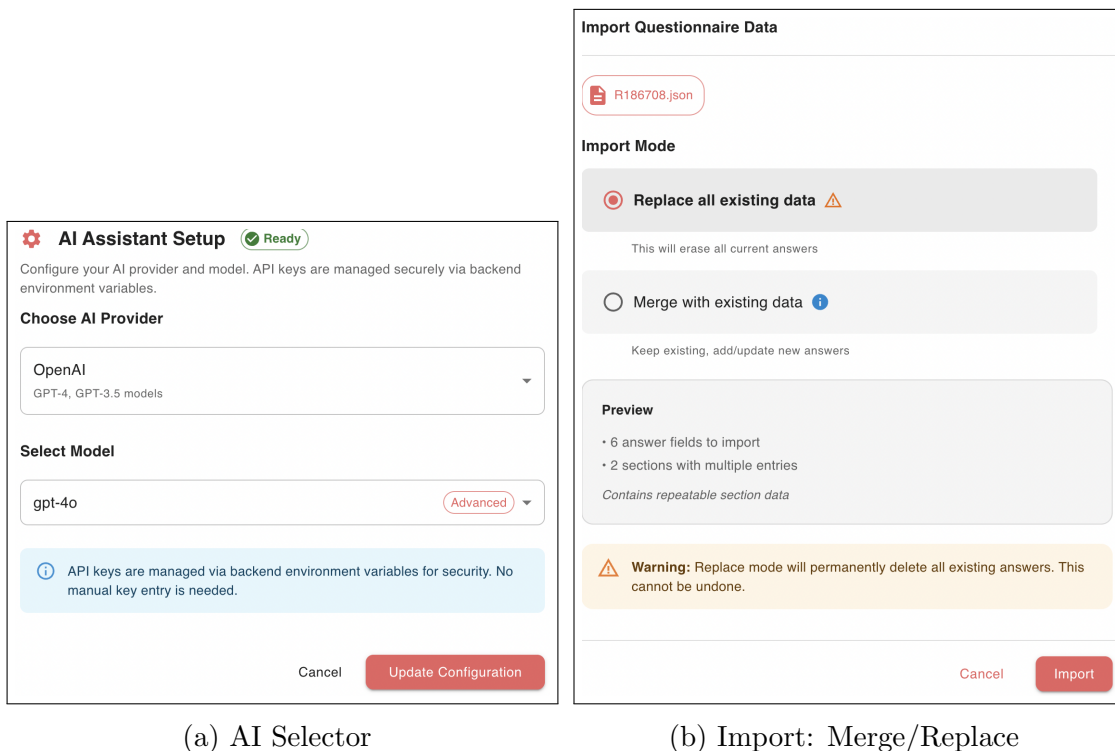
Three of the most frequent win-lose situations are shown in Table 1. Building a quick and sloppy product may be a low-cost, near-term "win" for the software developer and customer, but it will be a loss for the user (and the maintainer). Adding lots of marginally useful "bells and whistles" to a software product on a cost-plus contract may be a win for the developer and users, but is a loss for the customer. And "best and final offer" bidding wars imposed on competing developers by customers and users generally lead to low-ball winning bids which place the selected developer in a losing position.

Actually, nobody wins in the above situations. Quick and sloppy products destroy a developer's reputation and have to be redone, inevitably at a higher cost to the customer. The "bells and whistles" either disappear or (worse) crowd out more essential product capabilities as the customer's budgets are exhausted. Inadequate low-ball bids, imposed upon users, force them to accept a product

Figure 5.2: The unified split-screen workspace co-locating the hierarchical questionnaire and the PDF renderer. The sample document utilized for this system demonstration is [9].

2. **Data Import:** Facilitates the pre-filling of the questionnaire via a JSON file. Upon selection, an import dialog (Figure 5.3b) offers a choice between **merging** data with the current session or **replacing** it entirely. The dialog includes a structural preview:
 - *Example Preview:* 6 answer fields to import; 2 sections with multiple entries; Contains repeatable section data.
3. **Data Export:** Serializes the current state of the questionnaire into a structured JSON format for persistence or Knowledge Graph integration.
4. **Progress Dashboard:** A centralized validation hub (Figure 5.3c) containing three distinct status tabs: **Missing** (mandatory fields), **Invalid** (schema violations), and **AI Verification**. The latter allows the user to click "Verify All Answers" to generate a bulk quality-assurance report for the entire document (see the complete report interface in Appendix Figure A.2).

5.2. The Curation Workspace and User Interface



Validation Status		
MISSING (13)	INVALID (0)	AI VERIFICATION
Context		
What is the paper's DOI?		
In which venue series was the paper published?		
Research Questions & Answers		
What are the research questions of the study? #1 → Is the research question hidden in the running text?		
Is the answer to the research question explicitly highlighted in the paper?		
Is the answer to the research question hidden in the running text?		
Data Collection		
Which data collection methods did the study use? #1 → What is the type of this data collection method?		
What is reported about the collected data? → What is the type of the collected data?		
Data Analysis		

(c) Progress Dashboard: Missing, Invalid, and AI Verification tabs

Figure 5.3: Global session controls: model, import, and validation.

5.2.4 The AI Assist Interface

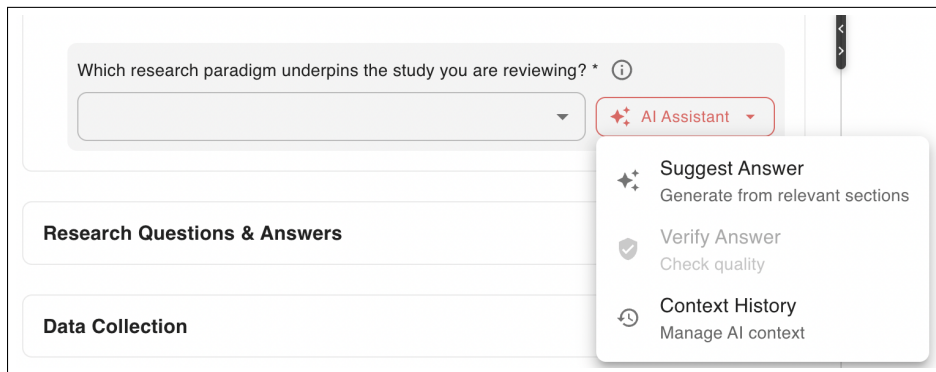
For every question in the questionnaire, the user can access a dedicated **AI Assist** menu. Upon activation, this interface (Figure 5.4a) provides a suite of interactive tools designed to assist with data extraction and verification:

1. **Suggest Answer:** This mode (Figure 5.4b) generates three ranked suggestions based on the document content. Each suggestion card displays a proposed value, a confidence score, and a verbatim evidence snippet. Selecting an evidence snippet automatically scrolls the PDF viewer to the referenced page and renders a spatial highlight over the source text (for a complete visual demonstration of this workflow, see Appendix Figure A.1). Users can directly populate a questionnaire field using the **Apply** button.
2. **Verify Answer:** This mode (Figure 5.4c) allows the AI to evaluate a manually entered answer. The system provides a feedback string, assigns a confidence rating, and retrieves supporting evidence from the PDF to validate or contradict the user’s input.
3. **Steering and Regeneration:** Positioned below the suggestions, a dedicated feedback field (Figure 5.5) allows users to provide natural language instructions to steer the model. By entering corrective feedback and clicking **Regenerate**, the user triggers a new suggestion cycle that incorporates these specific constraints into the next prompt payload. This iterative refinement process serves as the foundational “loop” within the system’s Human-in-the-Loop (HITL) architecture.
4. **History Manager:** This dialog (Figure 5.6) allows the user to curate the contextual memory of a specific question. It displays a chronological log of all previous AI interactions and the manual feedback provided during prior regeneration cycles. Users can explicitly include or exclude specific items from this log; excluded items are omitted from the prompt payload in subsequent requests to prevent past errors from influencing future results.

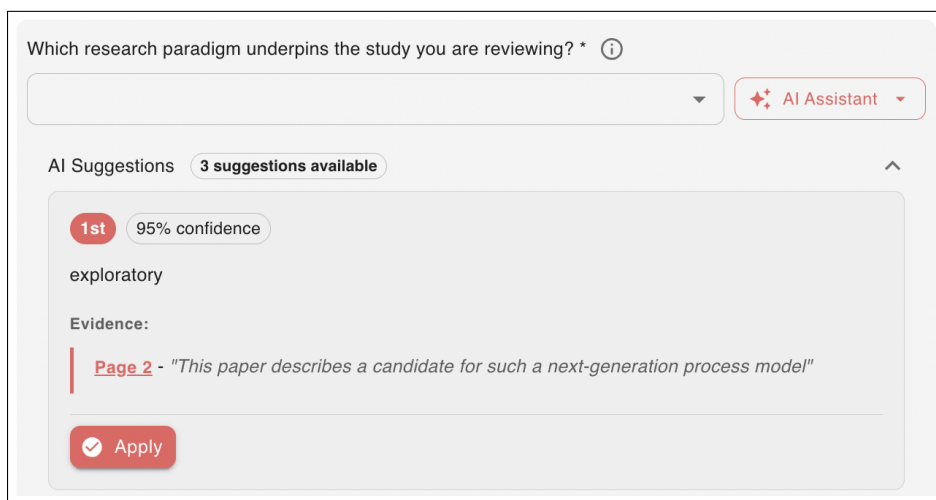
5.3 Implementing the Context Engine

The Context Engine serves as the core Retrieval-Augmented Generation (RAG) orchestrator for the extraction module. Unlike standard RAG pipelines that retrieve text based solely on isolated vector similarity, this engine is engineered to emulate the methodical approach of a human researcher. It achieves this by maintaining hierarchical awareness of the active schema, managing stateful chat

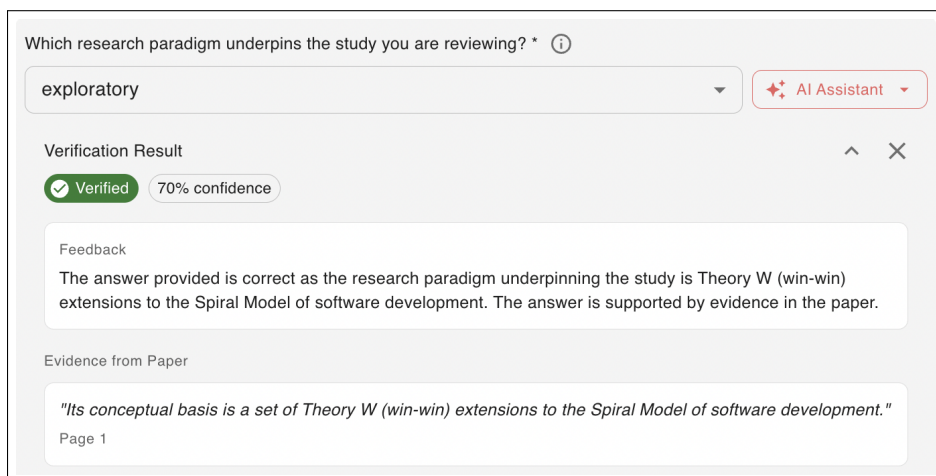
5.3. Implementing the Context Engine



(a) AI Assist menu options



(b) Ranked suggestions and evidence



(c) Verification feedback and evidence

Figure 5.4: AI Assist interface: menu and interaction modes.

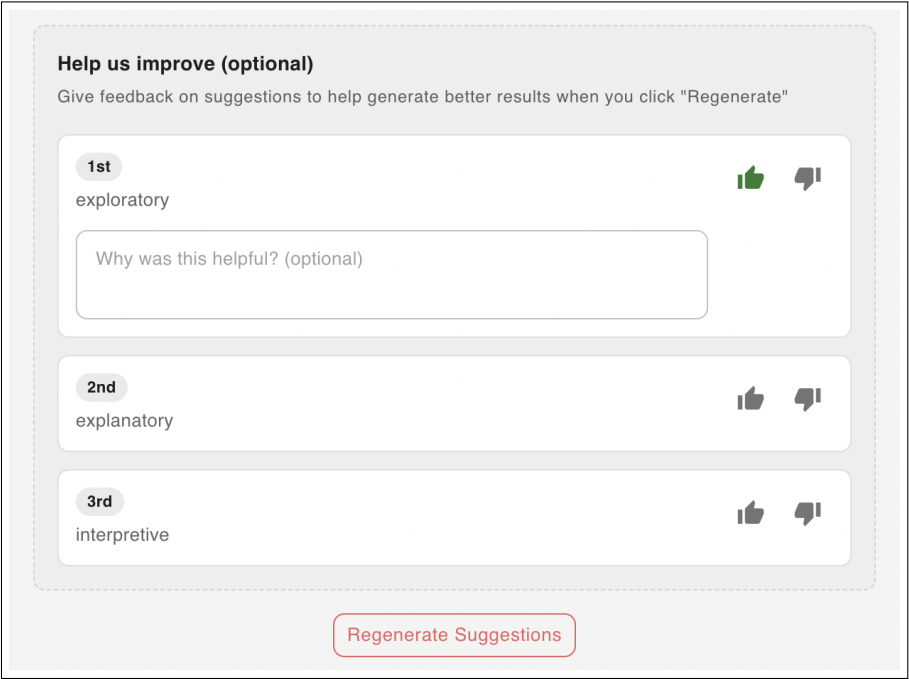


Figure 5.5: The feedback and steering interface.

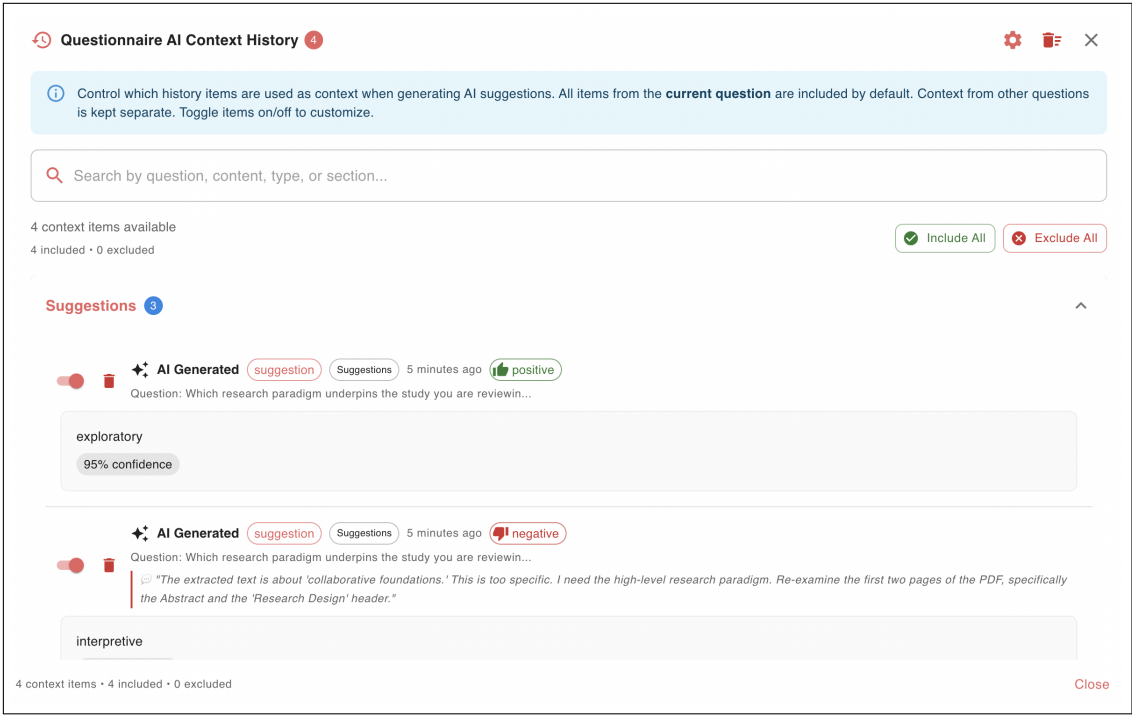


Figure 5.6: The History Manager for including or excluding past interactions.

history for iterative steering, and executing local vectorization to enforce strict data minimization.

Building upon the theoretical context types defined in Chapter 4, the following subsections detail the algorithmic implementation, data structures, and prompt engineering logic required to dynamically assemble and inject these varying contexts into the final LLM payload.

5.3.1 Transforming the PDF: Semantic Chunks and Vector Retrieval

Before the Context Engine can retrieve and inject evidence into the LLM prompt, the raw PDF content must be converted into semantically indexed chunks suitable for vector-based retrieval. This pipeline is the precise technical mechanism responsible for generating the *Source Context* (as defined in Chapter 4)—the grounded, factual text snippets retrieved from the manuscript. To successfully isolate and retrieve this Source Context, the foundational data pipeline executes two sequential phases: structural text chunking and local semantic vectorization.

Semantic Document Chunking

First, a custom client-side extraction layer processes the document page-by-page. Because raw PDF text extraction yields a fragmented sequence of text items, the algorithm reconstructs readable lines by using each item’s layout coordinates. Concretely, it groups items with similar vertical positions (i.e., the same rendered text line) and then concatenates them left-to-right to form coherent line strings, while still keeping a strict page boundary for every reconstructed block. This structured array of page texts is then dispatched to the backend.

Once received by the server, the text must be parsed into digestible segments to maximize retrieval accuracy. The backend implements a sentence-based chunking algorithm configured with a target chunk size of 2,000 characters (approximately 500 tokens). To prevent the arbitrary bisection of semantic meaning—which occurs when splitting strictly by character count—the algorithm specifically targets natural sentence boundaries. It achieves this by splitting the text using a positive lookbehind regular expression: `(?<=[.!?])(?=\s)`.

Finally, to ensure critical context is not lost between adjacent segments, the algorithm implements an explicit overlap strategy. The system dynamically prepends approximately the last 80 words from the preceding chunk. This word-based overlap ensures smooth semantic continuity across chunk boundaries, allowing the LLM to process fully contextualized text windows.

Local Vectorization and Architectural Trade-offs

As established in the conceptual Approach (Chapter 4), the Context Engine relies on mathematical vectorization and cosine similarity to retrieve relevant document chunks based on underlying meaning rather than rigid keyword matching. Translating this concept into a highly responsive production environment required navigating significant architectural trade-offs regarding the choice of the embedding model.

During the initial engineering phases, two standard approaches were evaluated and ultimately discarded due to significant operational inefficiencies:

- **The Cloud API Cost Matrix:** Relying on commercial, cloud-based embedding APIs (such as OpenAI’s embedding models) was preemptively ruled out to strictly control operational costs. Because the system’s final generation step already relies on external LLMs, introducing a secondary pay-per-token dependency just to vectorize the text would create an unsustainable cost matrix. Furthermore, maintaining local embeddings preserves the architectural flexibility to replace the generative LLM with a local, open-source alternative in the future, reducing dependence on third-party APIs and enabling a fully self-hosted deployment.
- **The Heavy Local Model Bottleneck:** To avoid cloud APIs, initial prototyping tested full-precision, local embedding models. However, these models proved computationally prohibitive. They exhibited unacceptably slow load times and high execution latency when running on standard server hardware without dedicated GPUs, creating severe bottlenecks during the document ingestion phase.

To resolve these inefficiencies, the architecture pivoted to a highly optimized, quantized solution. The system computes all vector embeddings locally using the `Xenova/all-MiniLM-L6-v2` ONNX model via the `@xenova/transformers` library [55, 54].

This specific model was selected because it successfully maps text chunks into 384-dimensional embeddings with high accuracy, yet its quantized nature reduces the total model size to approximately 22 MB. This tiny footprint allows the embedding model to be initialized and executed entirely within the backend Node.js process memory, bypassing the need for heavy compute instances.

During an extraction cycle, the semantic chunks are embedded dynamically on a per-request basis. The backend manually calculates the cosine similarity—utilizing the standard dot product over magnitude formula—across these temporary arrays in RAM. This purely in-memory execution strategy enforces data minimization, eliminates vector database hosting overhead, and drastically reduces overall infrastructure complexity.

5.3.2 Structural Context Assembly

While Chapter 4 defined the theoretical necessity of the Structural Context (the semantic “neighborhood”), translating this concept into a functional prompt requires precise state management. The extraction module must dynamically serialize the user’s active UI state into a structured text payload without overflowing the LLM’s context window.

The system efficiently gathers this context using React’s native top-down render cycle and flat dictionary lookups. This state aggregation is governed by specific token estimations (calculated at four characters per token) and item limits to optimize inference speed:

- **Parent Context (Top-Down State Propagation):** During the render cycle, higher-level groups pass down the relevant hierarchical state via prop drilling. This parent context is prioritized to keep the model grounded and avoid semantic ambiguity, and it is not truncated by token limits.
- **Sibling Context (Horizontal Dictionary Lookups):** To enforce horizontal consistency, the system queries a flat, centralized dictionary of the user’s current session answers. The implementation iterates through sibling IDs and retrieves the raw JSON values of up to five of the most recently answered questions at the exact same hierarchical depth (sorted by interaction timestamps). This aggregated state is rigorously truncated at a 500-token limit, ensuring the model receives sufficient adjacent facts without wasting computational resources on overly verbose peer data.
- **Adjacent Entry Context (Iterative Array Serialization):** For schema fields designed to accept multiple values (e.g., dynamically repeating question groups), the engine iterates through the prior entries contained within that specific array node. The algorithm fetches up to three previously logged entries and explicitly flags them in the prompt as already extracted. To maintain memory safety during highly repetitive extraction tasks, this sequential history is strictly bounded to a maximum of 900 tokens.

5.3.3 Implementing the HITL

The system implements a structured interaction log using a centralized React Context provider to maintain a persistent memory of the curation session. Each event is recorded as a typed object anchored to a specific `questionId`. This schema-driven approach ensures that the LLM’s context remains scoped to the active field, ensuring that historical feedback for one data point does not interfere with the extraction logic of unrelated fields.

A critical feature of this memory layer is the **Include/Exclude** mechanism. As illustrated in Figure 5.6, curators can manually toggle the `isExcluded` boolean for any historical entry. During the next regeneration cycle, the prompt engine performs a conditional filter on the state array, omitting any “excluded” items from the final payload. This allows the curator to prune the model’s memory, removing past hallucinations or irrelevant suggestions to maintain high reasoning quality within the model’s limited context window. The technical attributes of this schema are detailed in Table 5.1.

Field	Role
<code>type</code>	Event category (<code>suggestion</code> , <code>verification</code>).
<code>action</code>	Resulting state (e.g., <code>generated</code> , <code>applied</code> , <code>verified</code>).
<code>timestamp</code>	Temporal provenance for ordering and prompt injection.
<code>questionId</code>	Links the event to the corresponding questionnaire field.
<code>content</code>	Suggestion text or curator-provided answer.
<code>metadata.confidence</code>	Optional model confidence value.
<code>metadata.evidence[]</code>	Optional evidence snippets for verification and highlighting.
<code>metadata.feedback</code>	Optional curator rating/comment used for steering.

Table 5.1: Core fields of the interaction history schema.

5.4 Prompt Engineering and Data Validation

A primary structural limitation of LLMs within empirical data extraction is their inherent non-determinism. Unconstrained generative models frequently exhibit hallucination, structural drift, and conversational formatting rather than yielding parsable data objects. Because the frontend architecture relies on a strict data contract to populate the questionnaire state, any topological deviation from the expected JSON structure — such as conversational preambles or modified keys — interrupts the UI rendering cycle. To mitigate this vulnerability, the system introduces a highly constrained data transformation pipeline. This pipeline utilizes explicit prompt engineering, regex-based payload extraction, and a graceful fail-fast error handling architecture to enforce deterministic extraction without causing critical system crashes.

5.4.1 Designing the System Prompt

The foundation of generative accuracy within the pipeline is the static system prompt, which establishes absolute operational boundaries for the LLM. The prompt programmatically instructs the model to adopt a highly specific persona and adhere to a rigid set of extraction constraints. To systematically restrict generative creativity and ensure UI compatibility, the prompt enforces the following critical directives:

- The model must output precisely three suggestions, explicitly ranked by relevance and confidence.
- The model must extract exact text from the PDF, copying it word-for-word without adding external punctuation or paraphrasing.
- The model is instructed to utilize explicit contextual markers (e.g., [PAGE 3]) injected by the Context Engine to identify the exact page numbers for its evidence.
- Evidence excerpts are strictly constrained to 10-50 words to optimize the downstream highlighting algorithm.

Furthermore, the prompt explicitly defines the required JSON topology, strictly enforcing the keys required by the frontend parser, as demonstrated in Listing 5.1.

```
1 {  
2   "suggestions": [  
3     {  
4       "rank": 1,  
5       "text": "suggested answer",  
6       "confidence": 0.95,  
7       "evidence": [  
8         {  
9           "pageNumber": 3,  
10          "excerpt": "exact quote from the paper",  
11          "lineNumber": 4  
12        }  
13      ]  
14    }  
15  ]  
16 }
```

Listing 5.1: LLM Response Format

5.4.2 Dynamic User Prompt Assembly

While the static system prompt establishes the behavioral boundaries of the LLM, the actual extraction request relies on a dynamically assembled user prompt constructed by the client-side service layer. To synthesize the retrieved document chunks, the structural state of the questionnaire, and the temporal history of the curator’s feedback, the frontend programmatically concatenates a comprehensive text payload before dispatching the network request to the AI provider.

As illustrated in Figure 5.7, this assembly process acts as a programmatic data funnel, aggregating multiple disparate context streams into a single, cohesive LLM instruction block. The programmatic assembly conditionally integrates four distinct data layers into the final text payload, ensuring empty states do not cause unnecessary token bloat:

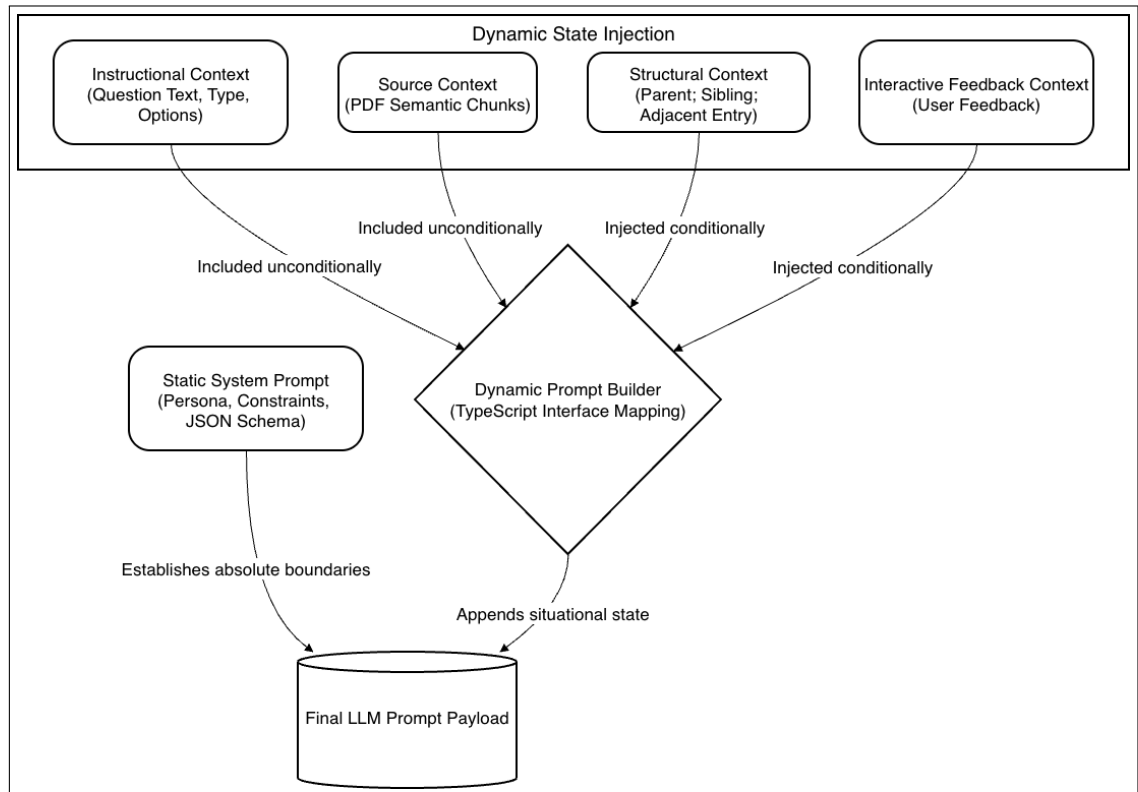


Figure 5.7: Programmatic data funnel aggregating context streams into a single LLM prompt.

- **Instructional Context:** By explicitly declaring question type and conditionally injecting the available options array, the system rigidly constrains the model’s vocabulary to match the application’s predefined database schema.

- **Source Context:** The semantic chunks retrieved by the local embedding model are injected under the “PDF Content” directive, serving as the sole grounded source of truth for the extraction task.
- **Structural Context:** To provide the LLM with the established facts of the surrounding questionnaire tree, the prompt conditionally appends the “PARENT CONTEXT” and “SIBLING CONTEXT”. Furthermore, to handle iterative, multi-value constraints, it injects “ADJACENT ENTRY CONTEXT”. Together, these three structural dimensions prevent the generation of contradictory empirical data and ensure sequential narrative consistency.
- **Interactive Feedback Context:** The inclusion of the active Human-in-the-Loop memory state under “PREVIOUS SUGGESTIONS” - Learn from History forces the AI to dynamically correct its trajectory based on prior human ratings and natural language feedback.

5.4.3 Regex Extraction and Fail-Fast Validation

Because LLMs occasionally prepend or append conversational text (e.g., “Here are the suggestions you requested”) to their JSON outputs, raw parsing attempts frequently fail. To resolve this, the client-side service layer executes a regular expression to surgically extract the JSON block from the encompassing text string before attempting native `JSON.parse()` execution.

Once the object is parsed in memory, the system executes a series of manual runtime checks to ensure topological correctness. As detailed in Listing 5.2, the system verifies the existence of the suggestions array, strictly truncates it to a maximum of three elements, and normalizes the underlying data fields with programmatic defaults (such as assigning a default confidence of 0.5 or an empty evidence array if omitted by the model).

```
1 // Extract JSON from response
2 const jsonMatch = result.text.match(/\{[\s\S]*\}/);
3 if (!jsonMatch) {
4   throw new Error('No JSON found in AI response');
5 }
6
7 const parsedResponse = JSON.parse(jsonMatch[0]);
8
9 // Validate structure
10 if (!parsedResponse.suggestions || !Array.isArray(
    parsedResponse.suggestions)) {
11   throw new Error('Expected suggestions array in response');
12 }
```

```

13
14 // Normalize suggestions
15 const suggestions = parsedResponse.suggestions
16   .slice(0, 3) // Ensure a maximum of 3
17   .map((suggestion: any, index: number) => ({
18     id: `suggestion-${Date.now()}-${index}`,
19     rank: suggestion.rank || index + 1,
20     text: suggestion.text || '',
21     confidence: suggestion.confidence || 0.5,
22     evidence: Array.isArray(suggestion.evidence) ?
23       suggestion.evidence : [],
24     createdAt: Date.now(),
25   }));

```

Listing 5.2: Regex-based JSON extraction and manual validation logic for LLM payloads.

Crucially, this architecture embraces a “fail-fast” methodology. If the regex extraction fails to locate a JSON block, or if the primary array structure is missing, the system immediately throws a discrete error rather than attempting complex, error-prone conversational scraping. By executing this validation directly within the frontend service layer, the system explicitly prevents corrupted or malformed data states from polluting the React component state and breaking the downstream highlight renderer.

5.5 The Spatial Search Engine and Visual Rendering

The frontend must physically locate and draw attention to the evidence text within the rendered document. Unlike standard web technologies (HTML) which structure text into continuous, semantic paragraphs, the Portable Document Format (PDF) is fundamentally a visual rendering canvas. A PDF document does not inherently “know” what a sentence or a paragraph is; instead, it renders text as disjointed, absolute-positioned nodes on a coordinate plane. Consequently, applying standard string-matching algorithms directly to a visual PDF is structurally impossible.

5.5.1 The PDF Document Object Model and Multi-Stage Mapping

To bridge this gap between visual rendering and programmatic text searching, the frontend architecture relies on Mozilla’s core rendering engine (`pdfjs-dist`) [37].

This engine parses the underlying binary of the PDF and exposes a Document Object Model (DOM) where text is represented as an array of disjointed nodes. Each node contains a raw string fragment—which could be a full word, a single letter, or a broken ligature—alongside a mathematical transform matrix that defines its exact geometric position, scale, and skew on the canvas.

The First Bridge: Flattening and the Reverse Index

Because LLMs generate continuous strings of text as evidence, the fragmented node architecture of the PDF must be temporarily flattened. When the document viewer loads a target page, the core extraction engine iterates through the entire array of PDF nodes, extracting the raw string from each and sequentially concatenating them into a single, continuous, searchable string.

However, this concatenation introduces a critical spatial disconnect. Once the text is flattened, the geometric coordinates of the individual words are inherently lost. If an algorithm finds a match at index 30 of the string, it has no way of knowing where index 30 physically lives on the screen.

To resolve this, the extraction algorithm simultaneously constructs a highly precise “reverse index” array in parallel. As it appends each individual character to the continuous string, it pushes a tracking object into the reverse index array. This tracking object records exactly which original PDF geometric node that specific character originated from. Because the continuous text string and the reverse index array are mathematically identical in length, every single character in the flattened string possesses a direct, 1-to-1 mapped relationship with its original spatial coordinate object.

The Normalization Problem and the Second Bridge

While the reverse index successfully maps a raw string back to the canvas, it introduces a secondary vulnerability: exact string matching is highly fragile. To ensure the search algorithm does not fail due to hidden artifacts, the system must aggressively “normalize” the text before searching—stripping out all spaces, punctuation, and non-alphanumeric characters.

However, modifying the text creates a severe structural breakdown. If the original flattened string is “AI Model” (8 characters) and the normalized string becomes “aimodel” (7 characters), the indices shift. Finding a match at index 2 of the normalized string no longer corresponds to index 2 of the original flattened string, effectively breaking the 1-to-1 reverse index mapping.

To prevent this architectural collapse, the system engineers a second structural bridge during the normalization phase. Utilizing a dedicated tracking function, the system generates a discrete position map that correlates the newly compressed

indices of the normalized string strictly back to their original indices in the un-normalized flattened string.

The Complete 3-Step Resolution Pipeline

By chaining these two structural bridges together, the system achieves a highly fault-tolerant spatial mapping pipeline capable of surviving aggressive data loss. When the LLM provides an evidence suggestion, the system executes the following resolution sequence:

1. **Search and Identify:** The algorithm searches the heavily compressed, normalized string and identifies the start and end indices of the match.
2. **Translation Phase 1 (The Position Map):** The system queries the position map, which translates the compressed normalized indices back into the original indices of the flattened text string.
3. **Translation Phase 2 (The Reverse Index):** The system queries the reverse index array with these restored indices, which instantly translates the one-dimensional string position back into a discrete list of two-dimensional PDF nodes.

This multi-stage pipeline ensures that the interface can aggressively alter, clean, and compress text for lightning-fast search capabilities, while flawlessly recovering the exact, unmodified geometric coordinates required to render the final visual highlight on the screen.

5.5.2 Execution of the Two-Pass Hybrid Search Algorithm

To computationally execute the hybrid search strategy and resolve the dual-layer OCR discrepancy established in Chapter 4, the system must parse the flattened text. To prevent UI blocking, the search space is first strictly bounded by injecting the page number variable extracted from the LLM JSON payload, successfully isolating the search loop to a single document page.

With the scope bounded, the implementation utilizes a custom, two-pass string matching strategy governed by a strict 50-millisecond execution timeout:

- **Pass 1 (Exact Match):** The algorithm first attempts a highly efficient, standard substring search. If the text is clean and the LLM extracted the evidence verbatim without any auto-correction, this pass succeeds instantly, preserving all original casing and punctuation.

- **Pass 2 (Aggressive Normalization):** If the exact match fails due to the aforementioned PDF artifacts, the system falls back to an aggressive mutation phase. The algorithm systematically strips away all whitespace, punctuation, special characters, and line breaks from both the target PDF string and the AI-generated search string, reducing them to pure, lowercase alphanumeric characters (strictly **a-z** and **0-9**).

By reducing a complex, artifact-heavy string like “Hello, World! [123]” down to a pure alphanumeric sequence like “helloworld123”, the algorithm computationally bypasses the most common PDF extraction errors at high speeds. When paired with the multi-stage position map, this two-pass execution yields highly accurate spatial highlights even across severe document breaks.

5.5.3 Geometric Bounding Boxes and Viewport Scaling

Once the search algorithm identifies the target PDF nodes, the system must render visual highlights over the document viewer. Since PDF canvases do not support standard browser text-selection APIs, the system calculates absolute-positioned bounding boxes by parsing the mathematical transformation matrix of each text node. This matrix defines the internal coordinate plane of the PDF, including position, scale, and skew. Because these values utilize an arbitrary internal coordinate system, a specialized conversion utility maps them into standard browser viewport pixels. This process includes the injection of minor padding to ensure visual clearance around the text fragment, allowing the evidence to be clearly framed within the UI.

To maintain the alignment of these highlights during browser zooming or split-screen resizing, the architecture employs hardware-accelerated CSS scaling rather than expensive JavaScript recalculations. All bounding box coordinates are calculated once against a fixed “base viewport” at a 1:1 scale ratio. When the user adjusts the zoom, the system calculates a single global scale factor and applies a CSS **transform: scale()** property to the parent container. Because the highlights are child DOM elements, they natively inherit this transform. This approach delegates the computational load of zoom-responsiveness to the browser’s optimized rendering engine, ensuring that geometric highlights remain flawlessly anchored to the source text without incurring the performance overhead of per-element coordinate math.

Chapter 6

Evaluation

To validate the architecture and interface described in the previous chapters, the system must be rigorously tested against authentic, domain-specific data. Evaluating the efficacy of the extraction assistant requires empirically measuring whether its AI-generated suggestions remain factually correct and reliable when processing complex, real-world scientific literature.

However, evaluating a steerable HITL system presents a unique methodological challenge. Because the system is specifically designed to interact with a human curator—allowing them to edit, reject, or steer the AI via natural language—the final output in the user interface is heavily influenced by subjective human behavior. To acquire objective, quantitative performance metrics, the evaluation framework must isolate the AI entirely. Therefore, this chapter details the creation of an independent, automated evaluation pipeline designed to test the system’s “zero-shot” extraction capabilities against verified ground-truth data, before any human interaction occurs.

6.1 Architectural Requirements for the Evaluation Framework

Before constructing the automated testing pipeline, strict architectural requirements were defined to ensure the resulting metrics would be scientifically valid and directly translatable to the production environment. The framework was engineered around three core pillars:

1. **Empirical Ground Truth (No Synthetic Test Cases):** The evaluation must not rely on simplified or synthetic examples. Instead, it must be grounded in a large corpus of real scientific publications paired with human-validated reference entries (i.e., previously extracted and verified answers).

2. **Architectural Parity with the Production Pipeline:** To ensure the metrics are representative of deployed behavior, the evaluation must not introduce a simplified surrogate pipeline. In particular, it should mirror the production extraction process as closely as possible, reusing the same PDF text-extraction logic, the same chunking strategy, and an equivalent prompt-construction procedure.
3. **Multi-Metric Comparison Strategies:** Because questionnaire fields vary in data type, the evaluation must apply type-aware grading logic rather than a single universal metric. The pipeline should programmatically detect the target type (e.g., boolean, categorical, or free text) and select an appropriate comparison strategy (e.g., exact match, normalization, semantic similarity, or set-based evaluation) to ensure fair scoring.

6.2 Evolution of the Evaluation Framework

Building a reliable, automated pipeline to process thousands of test cases while adhering to the strict requirement of “Architectural Parity” proved to be a significant engineering challenge. The architecture of the evaluation framework underwent three distinct evolutionary phases before achieving a reliable testing environment.

6.2.1 Phase 1: End-to-End Frontend Automation

The initial evaluation strategy attempted to benchmark the system using end-to-end (E2E) UI automation, simulating a human curator interacting directly with the React application. The testing script was designed to systematically upload each paper from the dataset, navigate the dynamic questionnaire, generate AI suggestions, and store the resulting data for evaluation. However, this approach was quickly abandoned due to the severe difficulty of scripting the interactions and the prohibitive time required for execution. First, developing robust automation scripts proved exceptionally complex due to the deeply nested, hierarchical structure of the questionnaire. Because repeating question groups are not mandatory by default, the automation script could not simply target static input fields. Instead, it had to programmatically navigate the nested schema, explicitly click “add item” buttons to instantiate new DOM elements for each array entry, and only then target the newly rendered AI suggestion triggers. Writing stable UI selectors for this multi-step, deeply nested DOM manipulation was highly error-prone and brittle. Second, even when the scripts executed successfully, routing the evaluation through the browser introduced massive temporal overhead. Simulating the complete user journey for thousands of extractions required uploading each individual PDF, waiting for the

client-side engine to parse and render the document, and waiting for visual UI state updates after every interaction. This UI-bound latency made processing the entire dataset exponentially time-consuming. Because of these dual bottlenecks, evaluating the underlying accuracy of the AI pipeline through the frontend was deemed unfeasible, necessitating a pivot to a direct, headless evaluation strategy.

6.2.2 Phase 2: Python-Based Logical Simulation

Realizing the evaluation needed to operate in a headless backend environment, the second approach attempted to simulate the extraction logic using Python, the industry-standard language for NLP evaluation. The goal was to write scripts that mimicked the frontend’s Context Engine and prompt construction.

However, after development, this approach was abandoned due to micro-deviations in the extraction pipeline. Differences in how Python PDF libraries extracted text compared to the JavaScript ecosystem, alongside subtle variations in how the scripts constructed the context payloads, meant the LLM was receiving slightly different inputs during testing than it would in production. If the evaluation payload is not highly faithful to the production payload, the resulting metrics are an inaccurate reflection of the live system.

6.2.3 Phase 3: Headless TypeScript Client and API Parity

To guarantee that the evaluation metrics reflected real-world production performance, the final architecture abandoned cross-language simulation in favor of a dedicated headless Node.js testing environment. The evaluation framework was engineered as a decoupled, automated client. This testing client faithfully reimplements the frontend’s Context Engine payload construction using locally scoped type definitions to ensure clean, isolated execution.

While architectural constraints required utilizing a Node-compatible PDF parser rather than the browser-bound Web Worker implementation used in production, the testing framework achieves strict parity at the backend level. The testing client dispatches the extracted data directly to the exact same production API endpoints responsible for semantic chunking and AI generation utilized by the live application.

By testing the true production backend infrastructure, the framework validates the exact semantic routing and AI generation logic used by human curators. Furthermore, to establish a pure baseline metric for the LLM’s inherent extraction capabilities, the system prompt was deliberately scoped for zero-shot evaluation, explicitly omitting the Human-in-the-Loop memory configurations (such as feedback history injection).

6.3 Automated Dataset Generation (The KG-EmpIRE Dataset)

Before any evaluation could occur, the framework required a vast repository of real-world test cases. Relying on manually created toy examples would invalidate the study’s claims of real-world efficacy. Therefore, an automated data-mining pipeline was engineered to dynamically construct a dataset of “PDF-to-JSON” pairs directly from the Open Research Knowledge Graph (ORKG).

Crucially, this extraction pipeline specifically targeted the KG-EmpIRE dataset. Because the literature within the KG-EmpIRE dataset was meticulously annotated and manually verified by domain experts—including the supervisor and examiner of this thesis—it does not merely represent a collection of test cases. Instead, it serves as a highly reliable, “gold-standard” benchmark for empirical requirements engineering curation, ensuring that the AI’s predictions are evaluated against the highest possible caliber of human academic reasoning.

6.3.1 SPARQL Data Extraction

To acquire this gold-standard ground-truth data, the pipeline executes an extensive SPARQL query against the ORKG triplestore endpoint. To systematically isolate the relevant literature, the query strictly filters for publications explicitly classified under the “Empirical Research” template (`orkgc:C27001`). Furthermore, to guarantee strict alignment with the curated KG-EmpIRE dataset, the query restricts the extraction exclusively to established software engineering venues: the IEEE International Requirements Engineering Conference (RE) and the International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ). By consolidating these constraints into a single, highly optimized query, the system successfully extracts the complete, nested questionnaire data for hundreds of papers simultaneously, perfectly preserving the integrity of the expert annotations.

6.3.2 Robust PDF Acquisition

With the ground-truth JSON data secured, the system must acquire the corresponding physical PDF documents to feed to the AI. Because scientific publishers frequently hide papers behind paywalls or use disjointed URL structures, a multi-strategy download pipeline was engineered.

The system first attempts to resolve the Digital Object Identifier (DOI) against specific publisher endpoints (such as IEEE, Springer, and ACM), executing custom scraping logic to bypass intermediate redirect pages. If the publisher restricts access,

the pipeline automatically falls back to querying the Unpaywall API to locate an Open Access version of the manuscript. Through this multi-tiered fallback strategy, the pipeline successfully downloaded and structurally verified the PDFs for 750 out of 776 queried metadata entries, achieving an exceptional 96.6% automated acquisition rate.

6.4 Multi-Metric Comparison Strategies

With the dataset generated, the core function of the evaluation framework is to compare the AI's generated suggestions against the human-verified ground truth. However, applying a universal, one-size-fits-all grading algorithm to an entire empirical questionnaire is mathematically flawed.

6.4.1 The Evaluation Challenge

Unlike traditional software testing where an expected output is strictly binary (correct or incorrect), evaluating LLM suggestions against complex scientific questionnaires introduces severe comparative complexities:

- **Semantic Paraphrasing:** An AI might extract “Machine learning enhances code quality,” while the human ground-truth states “Machine learning improves software quality.” While grammatically distinct, they are semantically identical.
- **Format Variations:** A boolean ground-truth might be stored as “false”, while the AI generates the string “no”.
- **Partial and Multi-Value Matches:** A research paper may contain three distinct data analysis methods. If the AI successfully extracts only one of the three, standard exact-match algorithms evaluate the entire response as a failure, stripping the AI of partial credit.

To account for these edge cases, the evaluation framework required a highly sophisticated comparison engine capable of judging underlying meaning and partial accuracy, rather than relying solely on brittle string matching.

6.4.2 Dynamic Metric Routing Engine

To resolve these discrepancies, the evaluation framework was engineered to dynamically read the target data type of the active question and route the comparison through one of four specialized grading algorithms:

- **Boolean Values (Normalized Match):** To prevent false negatives caused by format mismatches, the framework passes boolean answers through a normalization layer. Verbose LLM outputs (e.g., “Yes, the study reports...”) are stripped down using standard expressions to strict true/false values before applying an exact string match.
- **Single-Select Classification (Substring Fallback):** For strict multiple-choice options, the system first attempts a lowercase exact match. If there is no exact match, it falls back to a substring inclusion check, verifying if the ground truth is a substring of the predicted answer or vice versa.
- **Multi-Select Arrays (Maximized F1-Score):** For fields requiring multiple answers, grading an incomplete array as a binary failure is heavily penalizing. Instead, the framework evaluates the arrays as mathematical sets to calculate an F1-score. It runs three parallel matching approaches—exact string match, basic stemmed match, and a robust text extraction method that utilizes both stemming and compound word splitting to locate valid items anywhere within the prediction—and assigns the highest resulting score.
- **Abstract Text (Tri-Metric Similarity):** To evaluate free-text answers without penalizing valid paraphrasing, the system calculates three distinct scores and records the maximum value. It leverages BERTScore [56] for contextual embedding similarity, the Transformer model utilized in the chunking phase (all-MiniLM-L6-v2) [54] for sentence-level cosine similarity, and a containment check that applies both stemming and compound word splitting to verify if the core ground-truth terms are embedded within the generated text.

By employing these four specialized grading pathways, the evaluation framework successfully isolates the true reasoning capabilities of the AI, ensuring a fair, nuanced, and mathematically sound evaluation across highly diverse scientific data structures.

6.5 Resolving the Multiple Answers Discrepancy

While applying the correct mathematical metric solves the problem of format variation, the evaluation framework faced a secondary, structural challenge regarding how data is quantified in scientific literature.

6.5.1 The Challenge of Repeat Groups

In empirical research, certain data points are not isolated variables but belong to complex “repeat groups”—sets of interlinked questions that can possess multiple valid instances within a single publication. For example, a single paper might contain three distinct research questions. Consequently, the ground-truth dataset will contain three complete sets of answers for grouped variables, such as the exact phrasing of the research question, its specific classification type, and any associated sub-questions.

However, the EmpiRE-Compass AI assistant is fundamentally designed to assist the curator iteratively, extracting one discrete instance per interaction. If the AI is prompted to suggest an answer for a research question’s classification type, its goal is to identify a single, valid categorization for the specific research question it is currently targeting. It is structurally unfair—and contradictory to the system’s intended workflow—to expect the LLM to simultaneously output an array of all three classification types present in the broader paper. Yet, if the automated evaluation script naively compares the AI’s single valid string against the full ground-truth array containing all three instances, standard evaluation logic marks the prediction as a complete failure. This 1-to-many structural mismatch generated a massive influx of false negatives, heavily penalizing the AI’s strict metrics despite it successfully extracting a correct and highly relevant answer.

6.5.2 The Three-Tier Transformation Strategy

To prevent these false negatives and ensure the evaluation matched the actual intended behavior of the system, a deterministic transformation layer was engineered into the pipeline. This layer utilizes hardcoded structural rules to classify every specific extracted variable into one of three distinct evaluation tiers before any grading occurs:

- **Tier 1: Structured Repeating Groups (First-Item Evaluation):** For complex, nested entities such as Research Questions, the AI is expected to iterate through them one at a time. Therefore, the transformation layer intercepts the ground-truth array and isolates only the very first item. The AI’s single prediction is then compared exclusively against this first ground-truth item, ensuring a fair, 1-to-1 evaluation.
- **Tier 2: Simple Repeating Text (Max-Score Evaluation):** Unlike complex repeat groups, flat repeating fields (such as a list of specific statistical tests) do not require sequential grouping. Because the AI is designed to extract a single instance, the evaluation script compares the AI’s generated answer against *every* valid string within the ground-truth array. It then calculates

the match for each and records the maximum resulting score. This ensures that if the AI correctly extracts any valid item from the list, it is fully credited for a successful match, rather than being penalized simply for not selecting the very first item in the ground-truth array.

- **Tier 3: Multi-Select Arrays (Complete Set Evaluation):** Conversely, categorical checklist questions (such as “Data Analysis Types”) explicitly ask the user to select all applicable options simultaneously. For these specific questions, the transformation layer preserves the entire ground-truth array. The AI’s output is expected to be an array and is graded against the full ground-truth set using the Set-Based F1-Score metric.

By injecting this domain-specific transformation layer between the dataset extraction and the mathematical grading metrics, the framework successfully eliminated the false negatives caused by array mismatches. This guaranteed that the final accuracy metrics accurately reflected the AI’s performance within its defined operational constraints, rather than penalizing it for human curation formatting.

6.6 Experimental Setup and Execution

With the automated evaluation pipeline and dynamic grading metrics established, the framework was deployed against the KG-Empire dataset. To ensure a rigorous, fair, and economically viable assessment of the AI’s capabilities, several strict experimental parameters were defined prior to execution.

6.6.1 Dataset Subsampling and Statistical Convergence

While the automated acquisition pipeline successfully secured over 750 physical PDF documents, evaluating the entirety of this corpus through commercial LLM APIs incurs prohibitive computational and financial costs. Therefore, the experiment was designed as an incremental evaluation.

The testing framework sampled and processed papers from the dataset in batches. The execution was intentionally halted after processing 109 papers, yielding a total of 1,274 individually evaluated extraction questions. At this milestone, the accuracy metrics across all tested models had statistically converged—meaning the extraction success rates had stabilized and were no longer fluctuating with the addition of new data. Because continuing the evaluation beyond this point would simply consume API resources without altering the statistical outcome or providing novel insights, the sample size was deemed both scientifically robust and resource-efficient for this thesis.

6.6.2 The Top-1 Suggestion Constraint

In the current implementation, the AI assistant is explicitly designed to generate and present three distinct suggestions to the human curator. The rationale behind this UX design is to provide the user with multiple perspectives or interpretations of complex scientific text, empowering them to select the best direction and steer the AI via natural language feedback.

However, evaluating all three suggestions in an automated testing environment would artificially inflate the model’s success rate (i.e., grading a “Top-3 Accuracy”). To ensure the evaluation remains strictly conservative and does not overstate the AI’s inherent zero-shot capabilities, the primary analysis deliberately evaluates only the *first* (Top-1) suggestion returned by the LLM. If the first suggestion fails to meet the accuracy threshold, the extraction is marked as a failure, regardless of whether the correct answer was present in the second or third suggestion.

6.6.3 Thresholding for Binary Accuracy

To synthesize the results into a comprehensible “Overall Accuracy” percentage, the continuous scores generated by the dynamic routing engine must be converted into strict binary outcomes (Correct or Incorrect).

For Categorical and Boolean question types, this conversion is straightforward: the LLM’s output must achieve a perfect 1.0 exact or normalized match to be marked as correct. However, for the more complex data types, moderately strict thresholds were selected for this thesis to determine a passing grade. These values were chosen to balance strictness (penalizing clear mismatches) with practicality (avoiding over-penalizing minor phrasing variation and partial list extraction):

- **Multi-Select Arrays:** The set-based F1-score must be greater than or equal to “0.75”. This ensures the AI has extracted the vast majority of the required array items without generating excessive false positives.
- **Abstract Text:** The semantic similarity score must be greater than or equal to “0.70”. This moderately strict threshold guarantees that the AI’s generated text is highly semantically aligned with the human ground-truth, penalizing hallucinations while fairly accommodating natural paraphrasing.

Any extraction failing to meet these specific thresholds is strictly recorded as a 0.0 (Incorrect).

6.6.4 Model Baselines and Configurations

To definitively measure the impact of the context-aware architecture proposed in this thesis, the evaluation was executed across three distinct LLM configurations:

1. **GPT-3.5-Turbo:** Included as a legacy baseline to demonstrate the performance of older-generation generative models on complex scientific extraction.
2. **GPT-4o-mini (Zero-Shot Baseline):** Included as a modern, cost-effective baseline representative of current-generation LLM capability. In this configuration, the model is provided with the standard extraction prompt but operates entirely zero-shot, without any structural context injected.
3. **GPT-4o-mini + Ctx:** The primary experimental model. This configuration utilizes the exact same foundational model as the modern baseline but is dynamically injected with the structural “Parent” and “Sibling” context generated by the custom Context Engine detailed in the implementation.

To properly contextualize the behavioral shifts observed in the **GPT-4o-mini + Ctx** model, it is necessary to detail exactly how this relational data is formulated during the automated evaluation. The evaluation dataset maps to 25 distinct question fields within the empirical template. Of these, 17 specific fields are programmed to receive targeted, dynamically generated relational context.

Crucially, the evaluation framework mimics the exact sequential workflow of a human curator filling out the live questionnaire. Structural Context is not provided all at once; it is accumulated logically. For instance, if a logical grouping contains five sequential questions, the AI evaluates the first question entirely zero-shot, simulating the start of a curator’s task where no prior data exists. When evaluating the second question, the system dynamically injects the verified ground-truth answer from the first question as structural context. The third question then receives the context of both the first and second questions, creating a compounding chain of logical dependencies that accurately reflects the real-world curation process.

Additionally, a brief architectural distinction must be noted for this evaluation pipeline. While the primary application strictly separates the Parent and Sibling data streams to maintain a highly generic, domain-agnostic engine, the evaluation framework merges these concepts for simplicity. Because this specific evaluation script is strictly hardcoded to test the static KG-EmpIRE template—rather than adapting to dynamic, user-uploaded schemas—the technical boundary between a hierarchical parent node and a sequential sibling node is collapsed. Both types of relational data are simply merged into a single, targeted contextual payload prior to being injected into the prompt.

6.7 Evaluation Results

This section presents the objective outcomes of the automated evaluation pipeline. The final dataset comprised 1,274 independently evaluated extraction questions across 109 publications.

6.7.1 Overall Performance

The baseline results establish the foundational capabilities of the tested models, as summarized at the bottom of Table 6.1. The legacy GPT-3.5 baseline achieved an overall extraction accuracy of 40.4%. The modern, zero-shot GPT-4o-mini baseline improved this significantly, achieving a 54.9% overall accuracy. When the exact same GPT-4o-mini model was dynamically injected with the verified structural context (GPT-4o-mini + Ctx), the system’s overall Top-1 extraction accuracy increased to 58.8%.

Data Type	N	GPT-3.5	GPT-4o-mini	GPT-4o-mini + Ctx
Boolean	413	54.7% (226)	57.4% (237)	61.7% (255)
Multi-Select	379	29.3% (111)	59.4% (225)	60.7% (230)
Single-Select	254	44.9% (114)	62.6% (159)	59.4% (151)
Abstract Text	228	28.1% (64)	34.6% (79)	49.6% (113)
TOTAL	1,274	40.4% (515)	54.9% (700)	58.8% (749)

Table 6.1: Top-1 Suggestion Accuracy Aggregated by Question Type (N = 1,274)

6.7.2 Granular Findings

Breaking down the performance by specific data types (detailed in the upper rows of Table 6.1) reveals distinct directional trends. The injection of dynamic structural context yielded the highest absolute gain in Abstract Text variables, which experienced a massive leap from 34.6% to 49.6%. Boolean variables also saw a solid, stable increase from 57.4% to 61.7%. Multi-Select arrays experienced a minor but positive stabilization, rising from 59.4% to 60.7%. Conversely, Single-Select classification was the only data type to experience a performance regression, dropping from 62.6% to 59.4% when structural context was applied. The complete, granular breakdown of top-1 accuracy for every individual question ID is provided in Table 6.2 for reference. A detailed analysis of the performance disparities across these specific variables is presented in the Discussion.

6.7.3 Practical Utility (Top-3 Accuracy)

The preceding Top-1 evaluations strictly grade only the first suggestion generated by the AI. However, the implemented user interface is specifically designed to render three distinct suggestions side-by-side. To measure the practical efficiency of this design, the framework also calculated a “Top-3” metric, which records a success if any of the three generated suggestions met the passing thresholds.

6.7. Evaluation Results

Question ID	Data Type	N	GPT-3.5	GPT-4o-mini	GPT-4o-mini + Ctx
analysis_methods	multi_select	81	40.7% (33)	65.4% (53)	65.4% (53)
answer_hidden	boolean	109	23.9% (26)	92.7% (101)	92.7% (101)
data_type	multi_select	91	14.3% (13)	59.3% (54)	59.3% (54)
hypothesis_statement	text	4	75.0% (3)	75.0% (3)	100.0% (4)
hypothesis_type	single_select	4	75.0% (3)	75.0% (3)	100.0% (4)
measures_central	multi_select	42	21.4% (9)	59.5% (25)	52.4% (22)
measures_dispersion	multi_select	33	9.1% (3)	3.0% (1)	18.2% (6)
measures_frequency	multi_select	81	27.2% (22)	80.2% (65)	85.2% (69)
measures_position	multi_select	6	16.7% (1)	83.3% (5)	100.0% (6)
method_name_custom	text	91	3.3% (3)	13.2% (12)	49.5% (45)
method_type	single_select	91	46.2% (42)	41.8% (38)	41.8% (38)
ml_algorithms	text	17	52.9% (9)	64.7% (11)	64.7% (11)
ml_metrics	multi_select	12	66.7% (8)	91.7% (11)	75.0% (9)
other_analysis_methods	text	52	21.2% (11)	28.8% (15)	26.9% (14)
other_analysis_used	boolean	108	56.5% (61)	62.0% (67)	62.0% (67)
research_paradigm	single_select	109	34.9% (38)	73.4% (80)	71.6% (78)
rq_hidden	boolean	45	68.9% (31)	24.4% (11)	64.4% (29)
rq_highlighted	boolean	45	48.9% (22)	80.0% (36)	82.2% (37)
rq_text	text	45	64.4% (29)	53.3% (24)	53.3% (24)
rq_type	single_select	45	62.2% (28)	75.6% (34)	57.8% (26)
statistical_tests	text	15	46.7% (7)	80.0% (12)	73.3% (11)
subq_text	text	4	50.0% (2)	50.0% (2)	100.0% (4)
subq_type	single_select	5	60.0% (3)	80.0% (4)	100.0% (5)
threats_mentioned...	boolean	106	81.1% (86)	20.8% (22)	19.8% (21)
threats_reported	multi_select	33	66.7% (22)	33.3% (11)	33.3% (11)
TOTAL		1,274	40.4% (515)	54.9% (700)	58.8% (749)

Table 6.2: Top-1 Suggestion Accuracy by Granular Question ID (N = 1,274)

Under this relaxed metric, the legacy GPT-3.5 model achieved a 64.2% (818/1274) success rate. The zero-shot GPT-4o-mini baseline achieved 76.1% (969/1274). Ultimately, the contextually injected **GPT-4o-mini + Ctx** architecture reached a peak Top-3 accuracy of 77.9% (993/1274), meaning that in nearly 78% of all instances, the correct answer was placed visibly on the screen for the human curator.

Chapter 7

Discussion

While the raw evaluation metrics confirm that the Context Engine fundamentally alters the reasoning trajectory of the underlying LLM, a deeper analysis of the extraction logs is required to understand why these shifts occur. This chapter dissects the model’s behavioral patterns, analyzing exactly where contextual injection succeeds, detailing the inherent biases and evaluation bottlenecks of LLMs, and demonstrating why human steerability remains essential for empirical extraction.

7.1 The Impact of Contextual Injection

In empirical questionnaires, extraction tasks frequently operate in logical chains, where the extraction of one variable (e.g., identifying the specific text of a research question, `rq_text`) directly informs the evaluation of the next (e.g., determining whether that question was explicitly stated or implicitly embedded within the narrative, `rq_hidden`).

To evaluate the specific impact within the 17 fields augmented by structural context, the context-aware model (GPT-4o-mini + Ctx) demonstrated the following shifts compared to the zero-shot baseline:

- **Improvements (10 fields):** The majority of questions showed distinct accuracy gains. This was particularly evident in deeply chained fields where the model accumulates contextual history. For example, when extracting `measures_dispersion`, the model was dynamically injected with three preceding Siblings: the confirmation that descriptive statistics were used, followed by the specific measures of frequency and central tendency. This localized, compounding sequence rescued the model’s accuracy, driving it from a near-total zero-shot failure (3.0%) up to 18.2%. Similarly, fields that received extensive structural chains achieved perfect accuracy ceilings, with `measures_position`, `hypothesis_type`, and `subq_text` all jumping to 100.0%.

- **Maintained Performance (1 field):** The `ml_algorithms` field (64.7%) showed no change in accuracy. The injected Structural Context did not disrupt this extraction, proving that the Context Engine operates safely without introducing hallucination to this specific logical task.
- **Regressions (6 fields):** Six fields (`threats_mentioned_uncategorized`, `other_analysis_methods`, `measures_central`, `ml_metrics`, `rq_type`, and `statistical_tests`) experienced a drop in accuracy when Structural Context was applied. These regressions frequently occur due to a phenomenon known as algorithmic “overthinking.”

7.1.1 Algorithmic Overthinking

The most prominent example of contextual regression occurred in Single-Select classification. Deep analysis of the `rq_type` field, which plummeted from 75.6% to 57.8%, reveals how context can inadvertently cause the AI to overcomplicate a simple task.

Without context, the zero-shot model achieved an artificially inflated accuracy by simply defaulting to the majority class: “exploratory” (which comprises ~80% of the dataset). Conversely, injecting Structural context (including the exact text of the research question) forced the model to attempt a more nuanced classification. For example, given the specific research question *“How does accuracy vary by requirement type?”*, the zero-shot model correctly guessed the broad human ground-truth of “exploratory.” However, the context-aware model analyzed the injected phrasing “vary by type” and logically—but incorrectly according to the dataset’s overarching guidelines—classified it as “comparative.” The added context caused the model’s classification logic to become too granular, misaligning with the human annotators’ broader umbrella categories. Ultimately, this suggests that because the original manual data was heavily abstracted during collection, it is nearly impossible for the AI to identify the correct level of granularity required to match the human ground truth.

7.2 LLM Biases and Extraction Bottlenecks

Beyond the impact of context, the evaluation logs exposed several inherent limitations of foundational LLMs when applied to structured scientific extraction. These behavioral biases and evaluation bottlenecks manifest distinctly across different questionnaire data types.

7.2.1 Boolean Variables

The logs revealed a persistent behavioral bias in True/False extraction: the evaluated models exhibited a strong affirmative bias. When faced with methodological ambiguity in a paper, foundational models exhibit a strong tendency to guess “True” or “Yes.” This default behavior makes autonomously extracting true negative variables highly unreliable without human verification, as the AI prefers to assume a methodology was used rather than definitively confirming its absence.

7.2.2 Single-Select Classification

While the AI excels at finding evidence in a document, it frequently struggles to map that evidence to strict, human-defined single-select paradigms. This limitation is heavily driven by semantic keyword confusion. For instance, within the `method_type` field, if a paper mentions analyzing “online user reviews,” the LLM heavily associates the words “reviews” and “users” with a “survey” methodology. However, in the context of empirical software engineering, scraping existing online reviews is actually classified as “secondary research.” Similarly, the model frequently exhibits confusion between the terms “study” and “experiment,” treating them as interchangeable synonyms rather than distinct methodological categories. This proves that classifying text into a rigid empirical umbrella is a task that requires human domain expertise.

7.2.3 Multi-Select Arrays

Array-based extraction exposes a different behavioral flaw: “lazy generation.” Interestingly, a comparative analysis of the extraction logs reveals that this phenomenon was significantly more prominent in the modern **GPT-4o-mini** model than in the legacy **GPT-3.5** baseline. Even when explicitly prompted to extract an exhaustive multi-select array containing all relevant data points (such as types of data analysis described in the paper), the **GPT-4o-mini** model frequently outputs only the first or most prominent value it discovers in the text, ignoring the rest of the valid list. This artificial brevity severely penalizes the model’s strict set-based F1-score and practically mandates that human curators manually identify and append the missing secondary values.

7.2.4 Abstract Text Extraction

Evaluating abstract text extraction exposes severe limitations both in the LLM’s targeting capabilities and in the automated metrics used to grade it.

First, identifying specific narrative spans, such as `other_analysis_methods`, is inherently difficult for an LLM. Empirical papers are dense with analytical keywords and statistical jargon, making it highly challenging for the model to isolate the exact methodology name a human curator would select. Furthermore, upon consultation with the primary KG-EmpIRE curator, it was confirmed that locating exact author-defined formulations (such as those required for the `method_name_custom` field) is exceedingly difficult even for human experts, as it requires hunting for highly idiosyncratic phrasing rather than standard terminology.

Second, even when the LLM successfully locates the correct semantic concept, the automated evaluation pipeline frequently generates false negatives. Text extraction across the dataset remained heavily bottlenecked (peaking at 49.6%) primarily due to length and phrasing boundaries rather than absolute algorithmic failure. The evaluation framework utilized a robust three-strategy matching approach—combining strict substring containment, the Transformer model, and BERTScore. However, automated semantic math heavily penalizes length mismatches. If an evaluation ground-truth requires a specific two-word keyword, but the AI correctly extracts the entire, highly relevant sentence, the script often marks it as a failure because the extra words dilute the semantic similarity score. The AI successfully located the correct evidence within the paper, but failed to format it to the precise length anticipated by the script, proving that evaluating raw text autonomously is inherently flawed without human steerability.

7.3 The Necessity of Human-in-the-Loop (HITL) Steerability

The combination of inherent LLM biases, semantic confusion, and rigid evaluation bottlenecks directly dictates the architectural necessity of the Human-in-the-Loop (HITL) design. Fully autonomous pipelines are fundamentally unsafe for scholarly data curation because the AI cannot be trusted to resolve keyword ambiguities, overcome its affirmative biases, or format text to a curator’s exact subjective standard.

This limitation is especially evident in single-select classification questions where options share highly similar meanings (e.g., distinguishing a “survey” from “secondary research”), and in abstract text extraction where the LLM struggles with formatting boundaries. In these complex scenarios, the *evidence* highlighted by the LLM in the source document plays a significantly more valuable role than its final extracted answer.

By framing the AI as an “evidence hunter” rather than an autonomous decision-maker, these algorithmic bottlenecks are bypassed. The AI excels at instantly

scanning a 20-page PDF and locating the specific paragraph discussing user reviews or the sentence stating the hypothesis. The human curator then acts as the “classifier and refiner.” The human reads the located evidence and can instantly correct the AI’s naive categorical guess, or manually shorten a generated sentence down to the desired keyword.

Furthermore, human steerability is strictly required to counteract inherent behavioral flaws. While the evaluated model reliably exhibits an affirmative bias in Boolean questions and “lazy generation” in Multi-Select arrays, alternative foundational models will inevitably introduce their own distinct biases and failure modes. Therefore, active human steering remains the most robust, model-agnostic safeguard. A human curator must be present to dynamically steer the system—whether by forcing the current model to acknowledge true negatives and exhaustively hunt for secondary list values, or by correcting the unpredictable quirks of any future model deployed in the pipeline.

Observing the empirical results in conjunction with this behavioral analysis, it can be safely concluded that the proposed framework successfully addresses the core problem statement of this thesis. The system’s ability to generate three simultaneous suggestions in a single iteration—and successfully include the correct extraction among them in nearly 78% of all cases—demonstrates the high practical utility of the interface. By relying on the AI to surface concrete evidence and providing the human with direct steerability, the HITL interface effectively neutralizes the inherent flaws of autonomous pipelines, drastically accelerating manual data curation without sacrificing scientific rigor.

Chapter 8

Conclusion and Future Work

This final chapter synthesizes the core findings of the research and outlines the trajectory for future advancements. It begins by summarizing the key contributions of the framework in addressing the bottlenecks of scholarly data curation. Following this, the inherent limitations of the current architecture and evaluation methodology are critically examined. Finally, the chapter proposes concrete avenues for future work to further mature the system and expand its applicability across diverse scientific domains.

8.1 Conclusion

The transition of scholarly literature into structured Knowledge Graphs presents a significant bottleneck for researchers. While manual curation is inherently unscalable, relying on fully autonomous Large Language Models (LLMs) introduces critical risks to scientific rigor, including hallucinated evidence, semantic confusion, and inherent behavioral biases.

To address this gap, this thesis proposed a curator-focused LLM assistant, powered by a steerable Human-in-the-Loop (HITL) extraction framework and a flexible, model-agnostic Context Engine, designed to operate alongside the EmpiRE-Compass ecosystem. Rather than proposing a purely theoretical architecture, the primary objective was to build a highly practical, intelligent helper tool. This shift transitions the AI from an unpredictable, autonomous “black-box” into a targeted “evidence hunter” explicitly tailored to assist, rather than replace, human experts.

To justify this architecture, a rigorous empirical evaluation was conducted strictly on the underlying automated pipeline, comparing a zero-shot LLM baseline against the dynamically injected Context Engine. The results demonstrated that structural context successfully rescues the AI’s logical reasoning capabilities, drastically improving the extraction of dependent variables.

More importantly, the hard limits of this automated evaluation empirically proved the absolute necessity of the HITL paradigm. Because the autonomous extraction struggled with semantic keyword confusion and strict formatting boundaries, fully automated data entry remains unsafe for rigorous scientific reviews. However, by evaluating the model’s ability to generate multiple viable pathways, the results showed that the context-aware pipeline successfully produced the verified, correct answer within its top three suggestions in 77.9% of all interactions. While the live collaborative workflow and precise time-saving metrics remain to be tested in future user studies, this high Top-3 accuracy strongly validates the proposed architecture. It demonstrates that a curator-focused UI—designed to place multiple AI-generated options directly on the screen for a human expert to seamlessly accept, refine, or reject—is structurally primed to mitigate the inherent biases of foundational models and support rigorous scientific accuracy.

8.2 Limitations

While the proposed framework successfully optimizes the extraction workflow, several technical and architectural limitations must be acknowledged, particularly regarding the boundaries of the current automated evaluation and the system’s text-processing capabilities.

8.2.1 Exclusion of Multimodal Data

Currently, the parsing pipeline is restricted to textual data. Information embedded within images, figures, and complex tables is not considered during the chunking or extraction phases. While text-only extraction was sufficient for the specific KG-EmpIRE methodology template evaluated in this thesis, expanding this approach to a fully generic, domain-agnostic tool will require implementing advanced multimodal chunking strategies to accommodate visual data.

8.2.2 PDF Text Layer Artifacts and Highlighting Failures

A known technical limitation exists within the frontend evidence-highlighting algorithm due to the inherent structure of PDF documents. PDFs consist of a visual layer and an underlying text layer. Frequently, the underlying text layer contains OCR (Optical Character Recognition) artifacts or encoding errors (e.g., the visual word “will” being encoded as “wilf”).

When this flawed raw text is passed to the LLM, the model’s semantic understanding automatically corrects the spelling to “will” in its output. However, because the frontend relies on a highly efficient search algorithm to highlight the

evidence in the document viewer, it cannot find the corrected word “will” in the flawed raw text. This results in occasional highlighting failures. Addressing this would require implementing computationally heavy fuzzy-search algorithms; however, a deliberate architectural trade-off was made to prioritize system efficiency and speed over 100% highlighting coverage.

8.2.3 The Absence of a “No Value” Pathway

The system is heavily oriented toward an “answer-finding” approach and lacks a dedicated mechanism to explicitly declare when a value is absent from the source text. Because the architecture does not provide a designated “no value” or null-state output path, requesting a suggestion inherently forces the model to return a concrete extraction. Consequently, if a specific variable is genuinely not discussed in the publication, the system cannot autonomously output a true negative. This structural limitation requires the human curator to actively intervene and manually recognize when a generated suggestion is attempting to populate a missing field.

8.2.4 Limitations of Automated LLM Evaluation

Finally, a broader, industry-wide limitation encountered during this research is the inherent difficulty of automatically evaluating LLMs. Because LLMs are non-deterministic, their exact phrasing, verbosity, and structural output cannot be fully controlled. Evaluating these fluid responses using rigid programmatic logic or strict semantic metrics introduces unavoidable margins of error. Consequently, achieving absolute 100 percent precision in an automated evaluation pipeline is practically impossible, as otherwise correct semantic matches are frequently penalized for minor formatting or length deviations. While the definitive gold standard for assessing LLM extraction accuracy remains manual validation by a human expert, this approach is fundamentally unfeasible when benchmarking against large-scale empirical datasets, forcing a reliance on imperfect automated scripts.

8.3 Future Work

Based on the findings and limitations of this research, several promising avenues for future work have been identified to further mature the proposed framework.

8.3.1 Integration of Open-Source Local Models

While the current evaluation relies on proprietary OpenAI models (GPT-4o-mini), the architecture was explicitly designed to be model-agnostic. Future

iterations should benchmark the system using advanced open-weights models, such as DeepSeek. Transitioning to locally hosted models would eliminate data privacy concerns for unreleased research and significantly reduce long-term API costs. Crucially, eliminating these per-token financial barriers would grant researchers the operational flexibility to conduct much more extensive, iterative experimentation and pipeline tuning without prohibitive cost constraints.

8.3.2 Cross-Domain Generalization

To definitively prove the domain-agnostic capabilities of the schema engine, future work must evaluate the framework on different types of questionnaires and entirely distinct sets of scientific literature, such as biomedical or legal publications.

8.3.3 Direct Platform Integration via APIs

At present, the workflow still requires a curator to manually export the extracted results as a JSON file and then upload or re-enter them into external knowledge curation platforms such as ORKG. A practical next step is to integrate these platforms via their APIs so that curated entries can be pushed directly from EmpiRE-Compass into ORKG (or comparable systems), reducing friction, duplication of effort, and the risk of manual transfer errors.

8.3.4 Granular User Control and Multimodal Chunking

Currently, the retrieval system autonomously selects only textual chunks to send to the LLM, inherently excluding critical multimodal data such as figures, charts, and complex tables. Drawing inspiration from related works in interactive AI, a highly valuable UI feature would allow the curator to manually override and expand this selection. Empowering the user to explicitly add or remove specific document chunks from the LLM’s prompt window—including the ability to manually select and inject previously excluded visual or tabular data—would drastically enhance human steerability. This ensures that when automated textual retrieval fails, or when vital empirical evidence is locked within a diagram, the human expert can directly focus the multimodal AI’s attention exactly where it is needed.

8.3.5 Live User Studies and Workflow Validation

While automated metrics provide a foundational understanding of model accuracy, the ultimate value of the proposed framework lies in its ability to augment human workflows. Therefore, the logical next step for this research is to conduct live, timed user studies with human curators. Engaging domain experts in real-world

8.3. Future Work

extraction tasks will allow for the exact quantification of efficiency gains, such as the reduction in manual curation time per scholarly paper, and provide essential feedback to refine the human-in-the-loop interface.

Appendix A

Extended Interface Workflows

This appendix provides supplementary visual overviews of two critical Human-in-the-Loop (HITL) interaction pathways: the dynamic evidence highlighting system and the bulk verification pipeline.

A.1 Dynamic Evidence Highlighting

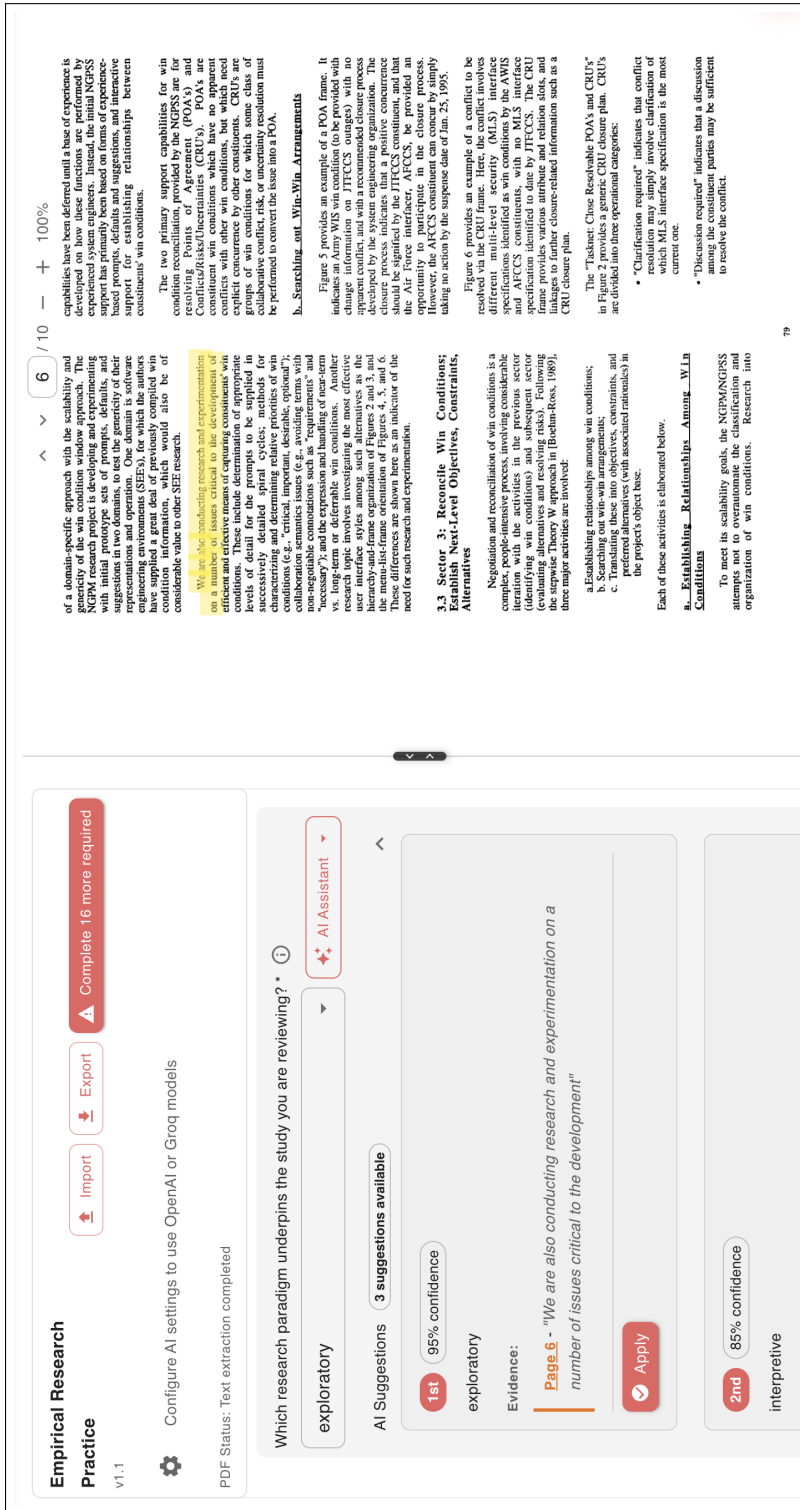


Figure A.1: The evidence highlighting feature. Clicking an evidence snippet dynamically locates and highlights the exact source bounding boxes within the PDF viewer, enabling instant verification.

A.2 Bulk Verification Pipeline

Validation Status		
MISSING (0)	INVALID (0)	AI VERIFICATION
✓ analysis_methods		75%
The answer provided is generally accurate. The paper discusses the use of descriptive statistics and inferential statistics in the data analysis process. While machine learning is not explicitly mentioned, it can be considered a subset of data analysis techniques.		
✓ descriptive_stats_used		75%
The answer is correct as the study does report descriptive statistics. However, to improve the quality of the response, specific examples of the descriptive statistics reported in the paper could be included.		
Suggestions:		
• Include specific examples of the descriptive statistics reported in the paper for a more detailed answer.		
⚠ inferential_stats_used		70%
The answer provided is partially correct. The study does report inferential statistics. However, the answer could be improved by providing more specific details and examples of the inferential statistics reported in the paper.		
Suggestions:		
• Provide specific examples of the inferential statistics reported in the paper.		
• Enhance the answer by mentioning the types of inferential statistics used and their relevance to the study.		
✓ ml_used		70%
The answer is correct. The study does report machine learning analysis.		
⚠ other_analysis_used		60%
The answer provided is not entirely accurate. The study does report other data analysis methods besides the ones mentioned. It is important to review the paper for a more comprehensive understanding.		
Close		

Figure A.2: The bulk verification report interface. Triggered via a single click, this feature executes an automated LLM verification cycle across the entire questionnaire, outputting a consolidated quality assurance report of all finalized answers

Bibliography

- [1] Abbi Abdel-Rehim et al. “Scientific hypothesis generation by large language models: laboratory validation in breast cancer treatment”. In: *Journal of the Royal Society Interface* 22.227 (2025).
- [2] Josh Achiam et al. “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [3] Amirreza Alasti, Lena John, and Oliver Karras. “From PDFs to Structured and FAIR Knowledge: Organizing NLP4RE ID Cards in the Open Research Knowledge Graph”. In: *Joint Proceedings of REFSQ-2026 Workshops, Doctoral Symposium, Posters & Tools Track, and Education and Training Track*. 2026.
- [4] Emily Alsentzer et al. *Publicly Available Clinical BERT Embeddings*. 2019. arXiv: [1904.03323](https://arxiv.org/abs/1904.03323) [cs.CL]. URL: <https://arxiv.org/abs/1904.03323>.
- [5] Sam Anzaroot et al. *Learning Soft Linear Constraints with Application to Citation Field Extraction*. 2014. arXiv: [1403.1349](https://arxiv.org/abs/1403.1349) [cs.CL]. URL: <https://arxiv.org/abs/1403.1349>.
- [6] Geetha Aradhyula. “Human-in-the-Loop Machine Learning Systems”. In: *World Journal of Advanced Engineering Technology and Sciences* 11.1 (2024), pp. 514–521. DOI: [10.30574/wjaets.2024.11.1.0012](https://doi.org/10.30574/wjaets.2024.11.1.0012).
- [7] Sören Auer et al. “Towards a knowledge graph for science”. In: *Proceedings of the 8th international conference on web intelligence, mining and semantics*. 2018, pp. 1–6.
- [8] Iz Beltagy, Kyle Lo, and Arman Cohan. “SciBERT: A Pretrained Language Model for Scientific Text”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui et al. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3615–3620. DOI: [10.18653/v1/D19-1371](https://doi.org/10.18653/v1/D19-1371). URL: <https://aclanthology.org/D19-1371/>.
- [9] B. Boehm et al. “Software requirements as negotiated win conditions”. In: *Proceedings of IEEE International Conference on Requirements Engineering*. 1994, pp. 74–83. DOI: [10.1109/ICRE.1994.292400](https://doi.org/10.1109/ICRE.1994.292400).
- [10] Lutz Bornmann and Rüdiger Mutz. “Growth rates of modern science: A bibliometric analysis”. In: *arXiv preprints* (2012).
- [11] Tek Raj Chhetri et al. “STRUCTSENSE: A Task-Agnostic Agentic Framework for Structured Information Extraction with Human-In-The-Loop Evaluation and Benchmarking”. In: *arXiv preprint arXiv:2507.03674* (2025).

-
- [12] Eli Cortez et al. “FLUX-CIM: flexible unsupervised extraction of citation metadata”. In: *Proceedings of the 7th ACM/IEEE-CS Joint Conference on Digital Libraries*. JCDL '07. Vancouver, BC, Canada: Association for Computing Machinery, 2007, pp. 215–224. ISBN: 9781595936448. DOI: [10.1145/1255175.1255219](https://doi.org/10.1145/1255175.1255219). URL: <https://doi.org/10.1145/1255175.1255219>.
 - [13] Danilo Dessì et al. “Cs-kg: A large-scale knowledge graph of research entities and claims in computer science”. In: *International semantic web conference*. Springer. 2022, pp. 678–696.
 - [14] Shehzaad Dhuliawala et al. “Chain-of-verification reduces hallucination in large language models”. In: *Findings of the association for computational linguistics: ACL 2024*. 2024, pp. 3563–3578.
 - [15] Vinicius Dos Santos et al. “Towards sustainability of systematic literature reviews”. In: *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 2021, pp. 1–6.
 - [16] Ayoub Harnoune et al. “BERT based clinical knowledge extraction for biomedical knowledge graph construction and analysis”. In: *Computer Methods and Programs in Biomedicine Update* 1 (2021), p. 100042.
 - [17] Kenji Hirohata et al. “Identifying sections in scientific abstracts using conditional random fields”. In: *Proceedings of the Third International Joint Conference on Natural Language Processing: Volume-I*. 2008.
 - [18] Aidan Hogan et al. “Knowledge Graphs”. In: *ACM Computing Surveys* 54.4 (July 2021), pp. 1–37. ISSN: 1557-7341. DOI: [10.1145/3447772](https://doi.org/10.1145/3447772). URL: <http://dx.doi.org/10.1145/3447772>.
 - [19] Hospice Hounbo and Robert E Mercer. “Method mention extraction from scientific research papers”. In: *Proceedings of COLING 2012*. 2012, pp. 1211–1222.
 - [20] Vinodh Ilango et al., eds. *Open Research Knowledge Graph*. Göttingen: Cuvillier Verlag, May 2024. ISBN: 978-3-689-52038-0. URL: <https://cuvillier.de/en/shop/publications/9037-open-research-knowledge-graph>.
 - [21] Mohamad Yaser Jaradeh et al. “Open research knowledge graph: Next generation infrastructure for semantic scholarly knowledge”. In: *Proceedings of the 10th international conference on knowledge capture*. 2019, pp. 243–246.
 - [22] Lena John et al. “ExtracTable: Human-in-the-Loop Transformation of Scientific Corpora into Structured Knowledge”. In: *International Conference on Theory and Practice of Digital Libraries*. Springer. 2026, pp. 470–487.
 - [23] Oliver Karras. “KG-EmpIRE: A Community-Maintainable Knowledge Graph for a Sustainable Literature Review on the State and Evolution of Empirical Research in Requirements Engineering”. In: *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. 2024, pp. 500–501. DOI: [10.1109/RE59067.2024.00063](https://doi.org/10.1109/RE59067.2024.00063).
 - [24] Oliver Karras et al. “Divide and Conquer the EmpiRE: A Community-Maintainable Knowledge Graph of Empirical Research in Requirements Engineering”. In: *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 2023, pp. 1–12. DOI: [10.1109/ESEM56168.2023.10304795](https://doi.org/10.1109/ESEM56168.2023.10304795).
 - [25] Oliver Karras et al. “EmpiRE-Compass: A Neuro-Symbolic Dashboard for Sustainable and Dynamic Knowledge Exploration, Synthesis, and Reuse”. In: *Joint Proceedings of REFSQ-2026 Workshops, Doctoral Symposium, Posters & Tools Track, and Education and Training Track*. 2026.

- [26] Selvani Deepthi Kavila and D. Fathima Rani. “Information extraction from research papers based on statistical methods”. In: *Proceedings of the Second International Conference on Computer and Communication Technologies: IC3T 2015, Volume 3*. Springer. 2015, pp. 573–580.
- [27] Holger Knublauch and Dimitris Kontokostas. “Shapes constraint language (SHACL)”. In: *W3C Candidate Recommendation* 11.8 (2017), p. 1.
- [28] Tomoki Kondo et al. “Technical trend analysis by analyzing research papers’ titles”. In: *Language and technology conference*. Springer. 2009, pp. 512–521.
- [29] Jinhyuk Lee et al. “BioBERT: a pre-trained biomedical language representation model for biomedical text mining”. In: *Bioinformatics* 36.4 (2020), pp. 1234–1240.
- [30] Sha Li et al. “Exploring LLMs for Scientific Information Extraction Using The SciEx Framework”. In: *arXiv preprint arXiv:2512.10004* (2025).
- [31] Zhijing Li et al. “Dual-LLM Adversarial Framework for Information Extraction from Research Literature”. In: *bioRxiv* (2025), pp. 2025–09.
- [32] Jimmy Lin et al. “Generative content models for structural analysis of medical abstracts”. In: *Proceedings of the hlt-naacl bionlp workshop on linking natural language and biology*. 2006, pp. 65–72.
- [33] Aixin Liu et al. “Deepseek-v3 technical report”. In: *arXiv preprint arXiv:2412.19437* (2024).
- [34] Nelson F Liu et al. “Lost in the middle: How language models use long contexts”. In: *Transactions of the association for computational linguistics* 12 (2024), pp. 157–173.
- [35] Meta Open Source. *React Documentation*. Accessed: 2026-03-26. 2026. URL: <https://react.dev/>.
- [36] Microsoft. *TypeScript Documentation*. Accessed: 2026-03-26. 2026. URL: <https://www.typescriptlang.org/docs/>.
- [37] Mozilla. *PDF.js (pdfjs-dist) Documentation and Source Repository*. Accessed: 2026-03-26. 2026. URL: <https://github.com/mozilla/pdf.js>.
- [38] MUI. *Material UI Documentation*. Accessed: 2026-03-26. 2026. URL: <https://mui.com/material-ui/getting-started/>.
- [39] Zara Nasar, Syed Waqar Jaffry, and Muhammad Kamran Malik. “Information extraction from scientific articles: a survey”. In: *Scientometrics* 117.3 (2018), pp. 1931–1990.
- [40] National Center for Science and Engineering Statistics (NCSES). *Publication Output by Region, Country, or Economy and by Scientific Field*. Science and Engineering Indicators (accessed: 2026-03-31). 2023. URL: <https://nces.nsf.gov/pubs/nsb202333/publication-output-by-region-country-or-economy-and-by-scientific-field>.
- [41] Zhen Ni and Hong Xu. “Automatic Citation Metadata Extraction Using Hidden Markov Models”. In: *2009 First International Conference on Information Science and Engineering*. 2009, pp. 802–805. DOI: [10.1109/ICISE.2009.353](https://doi.org/10.1109/ICISE.2009.353).
- [42] Overleaf. *Overleaf: Real-time Collaborative Writing and Publishing Tool*. Accessed: 2026-03-25. 2026. URL: <https://www.overleaf.com>.
- [43] Vipula Rawte, Amit Sheth, and Amitava Das. “A survey of hallucination in large foundation models”. In: *arXiv preprint arXiv:2309.05922* (2023).

-
- [44] Redux Team. *Redux Toolkit Documentation*. Accessed: 2026-03-26. 2026. URL: <https://redux-toolkit.js.org/>.
 - [45] Patrick Ruch et al. "Using argumentation to extract key sentences from biomedical abstracts". In: *International Journal of Medical Informatics* 76.2 (2007). Connecting Medical Informatics and Bio-Informatics - MIE 2005, pp. 195–200. ISSN: 1386-5056. DOI: <https://doi.org/10.1016/j.ijmedinf.2006.05.002>. URL: <https://www.sciencedirect.com/science/article/pii/S1386505606001183>.
 - [46] Jon Stephens, Shruti Sawant, and Amarnath Gupta. "Scientific Knowledge Graph Construction Needs an AI-Mediated, Scientist-in-the-Loop Workflow (A Blue Sky Paper)". In: *2025 IEEE International Conference on eScience (eScience)*. 2025, pp. 405–414. DOI: [10.1109/eScience65000.2025.00085](https://doi.org/10.1109/eScience65000.2025.00085).
 - [47] Markus Stocker et al. "SKG4EOSC-Scholarly Knowledge Graphs for EOSC: Establishing a backbone of knowledge graphs for FAIR Scholarly Information in EOSC". In: *Research Ideas and Outcomes* 8 (2022), e83789.
 - [48] Vercel. *AI SDK Documentation*. Accessed: 2026-03-26. 2026. URL: <https://sdk.vercel.ai/>.
 - [49] Ben Wagner, Johanne Kuebler, and Monika Zalnieriute. "Editorial: Humans in the loop: exploring the challenges of human participation in automated decision-making systems". In: *Frontiers in Political Science* Volume 7 - 2025 (2025). ISSN: 2673-3145. DOI: [10.3389/fpos.2025.1611563](https://doi.org/10.3389/fpos.2025.1611563). URL: <https://www.frontiersin.org/journals/political-science/articles/10.3389/fpos.2025.1611563>.
 - [50] Xingbo Wang et al. "SciDaSynth: Interactive Structured Data Extraction From Scientific Literature With Large Language Model". In: *Campbell Systematic Reviews* 21.4 (2025), e70073.
 - [51] Xingbo Wang et al. "Scidasynth: Interactive structured knowledge extraction and synthesis from scientific literature with large language model". In: *arXiv e-prints* (2024), arXiv–2404.
 - [52] Mark D Wilkinson et al. "The FAIR Guiding Principles for scientific data management and stewardship". In: *Scientific data* 3.1 (2016), pp. 1–9.
 - [53] Shican Wu et al. "Automated literature research and review-generation method based on large language models". In: *National Science Review* 12.6 (2025), nwaf169.
 - [54] Xenova. *all-MiniLM-L6-v2 Model Card (ONNX)*. Accessed: 2026-03-26. 2026. URL: <https://huggingface.co/Xenova/all-MiniLM-L6-v2>.
 - [55] Xenova. *Transformers.js Documentation and Source Repository*. Accessed: 2026-03-26. 2026. URL: <https://github.com/xenova/transformers.js>.
 - [56] Tianyi Zhang et al. "Bertscore: Evaluating text generation with bert". In: *arXiv preprint arXiv:1904.09675* (2019).
 - [57] Wayne Xin Zhao et al. "A survey of large language models". In: *arXiv preprint arXiv:2303.18223* 1.2 (2023), pp. 1–124.